# Efficient double-double matrix multiplication using the AVX-512 vector instruction set

**Alexander Sepelenco**

**A Dissertation**

Presented to the University of Dublin, Trinity College

in partial fulfilment of the requirements for the degree of

**Master in Computer Science**

Supervisor: Dr David Gregg

April 2025

# Declaration

I, the undersigned, declare that this work has not previously been submitted as an exercise for a degree at this, or any other University, and that unless otherwise stated, is my own work.

Alexander Sepelenco

April 18, 2025

# Efficient double-double matrix multiplication using the AVX-512 vector instruction set

Alexander Sepelenco, Master in Computer Science
University of Dublin, Trinity College, 2025

Supervisor: Dr David Gregg

Modern general purpose x86/x64 processors support two implementations of floating point arithmetic. These floating point numbers are called float or binary32, and double or binary64. In certain applications the precision of a double is not sufficient to give an accurate result. In such cases higher precision is required. Algorithms have been developed previously for emulating higher precision arithmetic known as double-double floating point precision. A double-double floating point number is the summation of two double precision numbers where the doubles represent a 64 bit high, and a 64 bit low precision. The 64 bit low precision number represents the next digits of the 128 bit number that cannot be represented in the high 64 bits. In this dissertation, I study the impact and analyse the performance of double-double precision matrix multiplication, utilising the AVX-512 vector instruction set.

This dissertation designs and implements highly efficient matrix multiplication algorithm optimising for register locality. Elements of existing matrix multiplication algorithms were adapted for use with double-double precision. Register locality, tiling, multithreading, data layout, and memory allocation were optimised to give a runtime performance of 3.93 clock cycles for single threaded execution and 3.27 nanoseconds per double-double multiply-add operation for multithreaded exeuction. An AVX-512 double-double multiply-add operation amounts to 28 floating point arithmetic instructions. Compared to a naive implementation of matrix multiplication without the optimisations of memory locality or register locality a 117 times improvement in runtime performance measured in seconds was shown for a 2048x2048 matrix on a Tiger Lake 11th generation Intel Core i7-1165G7, 2.8 GHz processor, a 4 core, 8 threaded processor. The optimised algorithm was determined to work best when using 4 threads.

# Acknowledgments

I would like to thank Dr David Gregg, my supervisor for his invaluable guidance and support throughout my dissertation. I am grateful to my parents for their encouragement throughout my degree.

Alexander Sepelenco

*University of Dublin, Trinity College*
*April 2025*

# Contents

# List of Tables

# List of Figures

# Chapter 1

# Introduction

## 1.1 Motivation

Matrix multiplication is a fundamental algorithm in mathematics. Matrix multiplication is used in linear algebra, a core part of mathematics and as such has a variety of applications. Practical applications involve calculating population movement, and analysis of traffic flow [2]. GNU MPFR [3], a library for multiple-precision floating-point computations can be used in matrix algorithms like the Krylov subspace method. The Kyrlov subspace method is a method of solving large linear systems. Previous research on performing quad-double precision on the Kyrolov subspace method by Hidehiko Hasegawa [4] is one example of the importance of enabling high precision matrix computation. The general importance of enabling high precision floating point matrix multiplication is therefore useful for other methods involving large linear systems with high precision. The hardware of general x86/x64 processors do not support floating point precision of 128 bits. General x86/x64 processors provide a hardware extension called x87 floating point for computing 80 bit floating point numbers. In cases where higher precision is required that exceeds 80 bits and requires 128 bits, a software implementation computing 128 bit floating point numbers using the summation of two double floating point numbers is referenced from the QD paper [5] and its corresponding library [6]. The implementation of double-double precision is modified to use the AVX-512 (Advanced Vector Extensions 512) vector instruction set to enable efficient double-double precision matrix multiplication.

Processors which support the AVX-512 vector instruction set have a fused multiply-add instruction. Floating point arithmetic operations are prone to rounding error. Below is a table with rounding errors that occur when using floating point numbers; see Table 1.1.

**Rounding in floating point arithmetic**

```
// Using multiply and add instruction without fused multiply-add
double mul = a * b; // rounding occurs
double mul_add = c + mul; // rounding occurs

// Using fused multiply-add instruction
double fmadd = fma(a, b, c); // rounding occurs
```

Table 1.1: The multiply and add instruction that do not use the fused multiply-add performs rounding after the multiply operation and addition operation. In total there is rounding for the two non multiply-add operation. In comparison, a fused multiply-add instruction performs rounding after the addition of the multiply-add instruction. The fused multiply-add totals to one floating point rounding computation.

AVX-512 compatible architectures support the fused multiply-add instruction. Using a fused multiply-add instructions enables a specific algorithm for computing double-double precision of a two-product number in which I discuss in Listing 4.3. The algorithm is reduced in complexity by exploiting the rounding of a fused multiply-add instruction. The QD paper [5] provides an alternative solution for computing the two-product of a double-double precision number without using the fused multiply-add instruction for processors which do not support it. The alternative algorithms developed in QD without the use of the fused multiply-add instruction is unnecessary for this dissertation as a fused multiply-add is available in the AVX-512 vector instruction set.

AVX-512 is more significant than providing a fused multiply-add instruction. Older vector instruction set implementations exist like AVX (Advanced Vector Extensions). Table 1.2 provides a summary on their main differences.

| | **Supports Fused Multiply-add** | **Register size** | **Total Register Count** |
|---|---|---|---|
| AVX-512 | Yes | 512 bits | 32 |
| AVX | Yes | 256 bits | 16 |

Table 1.2: AVX-512 and AVX comparison. AVX has twice as less total registers available per Central Processing Unit (CPU) core, and the size of the register is 256 bits rather than 512 bits for AVX-512.

## 1.2 Research Question

This dissertation explores different matrix multiplication algorithms with intermediate improvements found in known matrix multiplication algorithms. What optimisations create an efficient double-double matrix multiplication using the AVX-512 vector instruction set in runtime performance?

## 1.3 Research Objectives

1. Implement double-double arithmetic into matrix multiplication algorithms.
2. Modify existing double-double arithmetic functions to execute floating point arithmetic instructions in parallel using the AVX-512 vector instruction set.
3. Determine the impact of runtime performance using different data layouts of matrices for computing double-double floating point precision on matrix multiplication.
4. Explore existing implementations of efficient matrix multiplication to create a runtime performance of matrix multiplication comparable to finely tuned matrix multiplication algorithms.
5. Utilise the AVX-512 vector instruction set to improve runtime performance of matrix multiplication.

## 1.4 Outline

The following chapters cover:

1. Chapter 1 - Introduction. Presents the motivation, and highlights the technologies. The research question and a breakdown of objectives are detailed in this chapter.

2. Chapter 2 - State of the Art. A discussion on background provides a detailed introductory explanation of AVX-512 in Section 2.1.1, compilers in Section 2.1.2, floating point precision in Section 2.1.3, and matrix multiplication in Section 2.1.4.

3. Chapter 3 - Design. This chapter covers optimisation techniques that were implemented for register locality optimised algorithms by providing examples, figures and intuition for Listings 4.12, 4.13, 4.14, and 4.16.

4. Chapter 4 - Implementation. This chapter provides the AVX-512 optimised Listings 4.1, 4.2, 4.3, 4.4, and 4.6. General matrix multiplication in Listings 4.7, 4.8, 4.9, and 4.11 are presented. General matrix multiplication algorithms which are optimised for register locality like in Listings 4.12, 4.13, 4.14, and 4.16 are based off Chapter 3.

5. Chapter 5 - Evaluation. This chapter covers the results of the implementations by analysing the runtime performance of the algorithms from Chapter 4. Measurements of runtime in seconds in Figure 5.1, clock cycles per double-double multiply-add operation in Figure 5.5, and nanoseconds per double-double multiply-add operations in Figure 5.3 are detailed. Comparisons between different amount of threads and how the runtime is effected is presented in Figure 5.7.

6. Chapter 6 - Conclusions & Future Work. The chapter reviews the work completed up until this chapter. Final remarks are given on the dissertation and the solutions provided. The chapter discusses the limitation of the matrix multiplication solution provided in this dissertation in Section 6.1.1, and further evaluation that can be done on the algorithms provided in Section 6.1.3. Future work exploring impact of the AVX-512 vector instructions set on different processors in Section 6.1.2, utilising the upcoming AVX10 vector instruction set from Intel in Section 6.1.4, providing cross platform double-double precision matrix multiplication solutions for GPUs (Graphics Processor Unit) in Section 6.1.5, and expanding matrix multiplication to higher precision in Section 6.1.6 are all discussed in this chapter.

# Chapter 2

# State of the Art

In this chapter, I make reference to the key concepts and research on double-double precision and make reference research and information on floating point precision, AVX-512 and what makes the hardware so important for optimising on the CPU. I reference matrix multiplication, and compiler optimisations as well. These concepts are necessary in understanding later sections in this dissertation.

## 2.1 Background

In this section, I discuss the AVX-512 vector instruction set in more detail and present an example of programming using the instruction set. I discuss compilers and compiler flags in detail which are needed when dealing with floating point arithmetic, and the AVX-512 vector instruction set. Information on floating point arithmetic and how it correlates to double-double precision is explained. Finally, matrix multiplication with examples and code showcasing a naive implementation is presented.

### 2.1.1 AVX-512

AVX-512 is categorised under the term SIMD (Single Instruction Multiple Data), it utilises 512 bits to compute multiple data in parallel. SIMD is a form of parallel processing where multiple process elements perform the same operation on multiple data points simultaneously. AVX-512 is a type of acceleration which provides parallel computation for multiple elements. Intel categorises AVX-512 as "an essential workload-specific accelerator" [7]. SIMD is not limited to AVX-512; Other older implementations of SIMD exist, such as AVX (Advanced Vector Extensions) found in x86/x64 processors and uses 256 bits. Both Intel and AMD support this instruction set with recent generation chips from Intel disabling AVX-512 [8]. In future Intel intends to release AVX10 which extends AVX-512

with an official statement stating "Intel AVX10 extends and enhances the capabilities of Intel AVX-512" [9]. AVX10 has optional support for 512 bit wide vectors. This means that the work for this dissertation will see a performance improvement with no changes for AVX10 512 bit wide vectors. I discuss updating to AVX10 in Section 6.1.4. Unlike Intel, AMD's latest processors, Zen4 and Zen5 architecture have support for AVX-512 [10], [11]. AVX-512 is useful for its fused multiply-add and its wide registers that allow for parallel processing. The benefit of AVX-512 is and fused multiply-add is discussed in the motivation Section 1.1.

Writing AVX-512 code is different from writing sequential code. There are different ways of requesting the hardware to use SIMD instructions. SIMD can be written directly in assembly, or compiler intrinsics. Using intrinsics provides a higher level form of abstracting these instructions. Intrinsics are created by compilers following the hardware specification provided by the manufacturers of the CPU. Compliers implement intrinsics that enable efficient inline assembly that use the compilers optimisations which would not be possible if written in assembly. The GCC compiler provides SIMD intrinsics in the C header "immintrin.h". Intel provides a resource documenting SIMD instructions [1]. It is important to note that every function takes a different amount of compute time, so Intel specifies latencies and throughput in cycles per instruction (CPI) for a few of their recent architectures. The latency is measured in clock cycles and refers to the cycles it takes between the start and end of the instruction to execute. Modern processors support pipelining, a technique for implementing instruction-level parallelism. The technique is a way of allowing processors to execute multiple instructions in parallel while fetching the results of previous instructions. Latency therefore only gives information of how many cycles it would take to execute from start to finish if it were not pipelined. The CPI is also measured in clock cycles, but refers to how often the instruction is performed. Table 2.1 explains the throughput and latency of the 512 bit add double instruction.

| **Architecture** | **Latency** | **Throughput (CPI)** |
|---|---|---|
| Icelake Intel Core | 4 | 1 |
| Icelake Xeon | 4 | 0.5 |
| Sapphire Rapids | - | 0.5 |
| Skylake | 4 | 0.5 |

Table 2.1: Latency and Throughput for the "_mm512_add_pd" instruction in the Intel Intrinsics guide [1]. The Icelake Xeon core can deliver 2 add results per cycle with latency of 4 cycles.

Intel categorises certain intrinsics as "Sequence" instructions which are explained as intrinsics that generate "a sequence of instructions, which may perform worse than a native

instruction. Consider the performance impact of this intrinsic.". In practice this means that certain sequence instructions perform a collection of native instructions on the hardware. An example of a sequence instruction is the "_mm512_set1_pd". Hardware features of the processor will vary in what the compiler decides to generate as native instructions. This also means that the latency and throughput is hardware dependent and not available for sequence instructions as the compiler inserts a native collection of instructions to perform the sequence instruction. Table 2.2 provides an explanation of the difference between sequential code and AVX-512 code.

| **Sequential** | **AVX-512** |
|---|---|
| c[0] = a[0] + b[0] | _mm512d a8 = _mm512_loadu_pd(&a[0]); |
| c[1] = a[1] + b[1] | _mm512d b8 = _mm512_loadu_pd(&b[0]); |
| c[2] = a[2] + b[2] | _mm512_storeu_pd(&c[0], _mm512_add_pd(a8, b8)); |
| c[3] = a[3] + b[3] | |
| c[4] = a[4] + b[4] | |
| c[5] = a[5] + b[5] | |
| c[6] = a[6] + b[6] | |
| c[7] = a[7] + b[7] | |

Table 2.2: Comparison between sequential code and AVX-512 code. The difference in these implementations is that the sequential is loading in 16 values, performing eight arithmetic instruction and storing eight values. In comparison the AVX-512 code performs two loads, one arithmetic instruction, and one store.

### 2.1.2 Compilers

As stated in the previous section, compilers implement intrinsics. The compiler I chose to target is GCC, a mature C compiler, but other compilers will work as long as the equivalent flags are chosen for the compiler. In this section, I talk about the Clang compiler and give the equivalent flags required to ensure the implementation of the algorithms are correct. Systems level programming language such as C/C++ are commonly used for optimising and targeting hardware. The GCC compiler version used for this dissertation is v14.2.1, and the Clang version I referenced is v14.0. These versions are relevant for the section below discussing feature flags provided by GCC and Clang as compiler flags may change in future for older or newer versions of the compiler.

#### 2.1.2.1 Compiler Flags

Compilers often time create features and optionally allow users to enable the feature with a flag. If a flag provides an important feature, a compiler may turn it on by default [12].

In GCC C99 and above, the standard enables the fused multiply-add instruction, an important feature which AVX-512 supports. If targeting Clang, a version of v14.0 or greater must be used or the flag "-ffp-contract=fast" needs to be enabled [13].

Compilers by default do not specifically target an architecture or all hardware features by default. This means that GCC requires an additional flag. If compiling locally with a processor supporting AVX-512, both GCC and Clang compilers allow for the flag "-march=native", which will enable a list of flags most useful for the architecture its targeting. If compiling for a generic machine it may be necessary to use the flag "-mavx512f" to enable AVX-512F (Advanecd Vector Extension 512 Foundation) [14] , but the other flags enabled by "-march=native" will not be present and so targeting the specific hardware is no longer possible. AVX-512F is the name of the extension set that enables the 512 bit wide vectors. In assembly they are referred to as "zmm" registers. AVX-512 has different extensions which enable different features of AVX-512, so to utilise the most out of AVX-512, the 512 bit registers are useful to have hence the AVX-512F flag being enabled.

For consistency in testing the runtime performance an additional flag "-fno-tree-vectorize" in GCC was enabled. For Clang the comparable flags are "-fno-slp-vectorize" and "-fno-vectorize". Both GCC and Clang will automatically enable vectorisation if the "-O3" optimisation is enabled. For GCC and Clang the flag "-O3" will optimize while retaining the IEEE-754 standard for floating point arithmetic [15]. By disabling the vectorisation flags, I eliminate the issue of analysing the performance of code that differs from the code that I have written. The reasoning for disabling automatic vectorisation is to prevent future compiler versions from altering the code generation of the algorithms discussed in this dissertation. An objective for the dissertation is to provide a resource in the design and implementation of matrix multiplication algorithms. If I were to have auto vectorized code then the algorithms 4.7, 4.8, 4.9, 4.13 which I intended to not have SIMD do in fact have SIMD. This dissertation took the specific algorithms which were suitable for auto vectorisation and provided a comparison between the runtime performance impact AVX-512 code has on efficient code generation. The algorithms that were not optimised using SIMD were optimised manually as separate functions using AVX-512 for Listings 4.10 and 4.14.

### 2.1.3 Floating point arithmetic

In this section, I discuss floating point arithmetic, and talk about the IEEE-754 standard, the compiler features for higher precision floating point arithmetic, and double-double precision numbers followed by a summary of the data types.

#### 2.1.3.1 IEEE-754

IEEE-754 is a standard encoding floating point arithmetic [15]. It defines how numbers are represented like float or binary32, double or binary64, and quadruple precision or binary128.

#### 2.1.3.2 Compiler specific 128 bit floating extensions

The GCC and Clang compiler provides an extension for enabling 128 bit floating point precision without requiring the use of two doubles. This data type is called "__float128". Despite the compiler enabling 128 bit floating point arithmetic, there is not hardware support for general x86/x64 processors. This means that this floating point number is implemented in software rather than hardware. AVX-512 does not support loading and performing operations on 128 bit floating values. AVX-512 only supports 32 bit and 64 bit floating point operations. Therefore the extension present in the GCC and Clang will not be used for computing 128 bit floating arithmetic as this dissertation requires the use of the AVX-512 vector instruction set.

#### 2.1.3.3 Double-Double precision

Double-Double floating point precision is the summation of two 64 bit floating point numbers a *hi* and a *lo*. The *lo* contains the result of the next least significant bits after the 64 bit of *hi*. Since double-double precision uses two double values to represent the 128 bit floating point number, the sign is 1, exponent is the same as a double which is 11 bits, and the mantissa based on the QD libraries implementation is 109 bits. This makes the data type double-double slightly less precise than a quadruple data type. Table 2.3 represents a double-double precision number.

| **64 bit** | **64 bit** |
|---|---|
| *hi* | *lo* |

Table 2.3: Double-Double floating point precision number representation

The QD library [6] implements double-double and quad-double precision from the research paper by Yozo Hida, Xiaoye S. Li and David H. Bailey [5]. It is written in C++ and does not include the AVX-512 optimisations. Quad-Double precision defined in the QD library is not required as the focus for this dissertation is to create a solution for double-double precision rather than quad-double precision. Future work on creating a quad-double solution is discussed in the conclusion & future work chapter under Section 6.1.6. Table 2.4 shows the differences between floating point data types, their hardware and AVX-512 support.

| **Data Type** | **Sign** | **Exponent** | **Mantissa** | **Hardware?** | **AVX-512?** |
|---|---|---|---|---|---|
| **float** | 1 bit | 8 bits | 24 bits | Yes | Yes |
| **double** | 1 bit | 11 bits | 53 bits | Yes | Yes |
| **quadruple** | 1 bit | 15 bits | 113 bits | No | No |
| **double-double** | 1 bit | 11 bits | 109 bits | No | Yes |

Table 2.4: Floating point data types. The quadruple precision floating point data type is also called __float128 in GCC and Clang extensions and does not support AVX-512.

The implementation in the QD library of double-double floating point precision is written in C++ without the optimisations of AVX-512. Matrix multiplication requires addition and multiplication operations and only those operations are taken from the QD library. The product of a double-double number can also be computed using a fused multiply-add or without one because AVX-512 supports a fused multiply-add instruction and the algorithms from QD have an implementation of multiplication using fused multiply-add; see Listing 2.3. The algorithms in the QD paper which are required are algorithms 3, algorithm 4, and algorithm 7. The algorithms take the *hi*, and *lo* 64 bits of a double as their parameters. In the QD paper the doubles are represented as "a" for *hi* and "b" for *lo*. Algorithms 3, 4, and 7 are taken from the QD paper performing operations on a double-double number, with their equivalent code modified into C. Listings 2.1, 2.2, and 2.3 are the equivalent helper functions performing operations on a double-double number. They are used in functions that add and multiple two double-double numbers; see Listings 2.4 and 2.5. The functions for computing a double-double precision number using the AVX-512 vector instruction set is listed in the implementation chapter; see Section 4.1. The operations are assumed to be performed in IEEE double format, with rounding. For any binary operator $\cdot \in \{+, -, \times, /\}$ then $\text{fl}(a \cdot b) = a \odot b$ denotes the floating point result of $a \cdot b$, and the definition of $\text{err}(a \cdot b)$ is $a \cdot b = \text{fl}(a \cdot b) + \text{err}(a \cdot b)$.

**Algorithm 3** QUICK-TWO-SUM$(a, b)$ from the QD paper [5]
The following algorithm computes $s = \text{fl}(a + b)$ and $e = \text{err}(a + b)$ assuming $|a| \geq |b|$.

1: $s \leftarrow a \oplus b$
2: $e \leftarrow b \ominus (a \ominus a)$
3: **return** $(s, e)$

Listing 2.1 implements Algorithm 3, the quick summation operation of a double-double precision number. This function is used as a helper function in addition; see Listing 2.4. A detailed explanation is provided for the AVX-512 equivalent implementation in Listing 4.1.

```
double qd_dd_quick_two_sum(double a, double b, double *err) {
    double s = a + b;
    *err = b - (s - a);
    return s;
}
```

Listing 2.1: C equivalent code of Quick-Two-Sum modified from the QD library [6].

**Algorithm 4** TWO-SUM$(a, b)$ from the QD paper [5]
The following algorithm computes $s = \text{fl}(a + b)$ and $e = \text{err}(a + b)$. This algorithm uses three more floating point operations instead of a branch.

1: $s \leftarrow a \oplus b$
2: $v \leftarrow s \ominus a$
3: $e \leftarrow (a \ominus (s \ominus v)) \oplus (b \ominus v)$
4: **return** $(s, e)$

Listing 2.2 implements Algorithm 4, the summation operation of a double-double precision number. This function is used as a helper function in addition; see Listing 2.4. A detailed explanation is provided for the AVX-512 equivalent implementation in Listing 4.2.

```
double qd_dd_two_sum(double a, double b, double *err) {
    double s = a + b;
    double bb = s - a;
    *err = (a - (s - bb)) + (b - bb);
    return s;
}
```

Listing 2.2: C equivalent code of Two-Sum modified from the QD library [6].

**Algorithm 7** TWO-PROD$(a, b)$ from the QD paper [5]
The following algorithm computes $p = \text{fl}(a \times b)$ and $e = \text{err}(a \times b)$ on a machine with an FMA (Fused Multiply-Add) instruction. Line 2 of the algorithm is using $\times$ of $\otimes$ and $-$ instead of $\ominus$ because the fused multiply-add instruction performs rounding once after both operations. After performing the fused multiply-add then rounding occurs.

1: $p \leftarrow a \otimes b$
2: $e \leftarrow \text{fl}(a \times b - p)$
3: **return** $(p, e)$

Listing 2.3 implements Algorithm 7 for a fused multiply-add operation of the product of a double-double precision number. This function is used as a helper function in multiplication; see Listing 2.5. A detailed explanation is provided for the AVX-512 equivalent implementation in Listing 4.3.

```
double qd_dd_two_prod(double a, double b, double *err) {
    double p = a * b;
    *err = __builtin_fma(a, b, -p);
    return p;
}
```

Listing 2.3: C equivalent code of Two-Prod explicitly using the fused multiply instruction from built in GCC modified from the QD library [6].

The QD library also provides functions for adding and multiplying double-double against a double-double [6]. Listings 2.4 adds two double-double numbers and returns the *hi* precision and *lo* precision values. A detailed explanation is provided for the AVX-512 equivalent implementation in Listing 4.4.

```
void qd_dd_ieee_add(double a_hi, double a_lo,
                    double b_hi, double b_lo,
                    double *hi, double *lo) {
    /* This one satisfies IEEE style error bound,
       due to K. Briggs and W. Kahan.                    */
    double c_hi, c_lo, c_hi_err, c_lo_err;

    c_hi = qd_dd_two_sum(a_hi, b_hi, &c_lo);
    c_hi_err = qd_dd_two_sum(a_lo, b_lo, &c_lo_err);
    c_lo += c_hi_err;

    c_hi = qd_dd_quick_two_sum(c_hi, c_lo, &c_lo);
    c_lo += c_lo_err;
    c_hi = qd_dd_quick_two_sum(c_hi, c_lo, &c_lo);

    /* return */ *hi = c_hi; *lo = c_lo;
}
```

Listing 2.4: Double-Double addition modified from the QD library [6]. The "qd_dd_two_sum" function adds two double types and propagates the error term from the parameters. The "qd_dd_quick_two_sum will correct the term. The precondition to the "qd_dd_quick_two_sum" is $c\_hi \geq c\_lo$. The calls to the quick sum ensures the return values of *c_hi* and *c_lo* are represented as one complete double-double precision number satisfying the IEEE style error bound.

Listings 2.5 multiplies two double-double numbers and returns the *hi* precision and *lo* precision values. A detailed explanation is provided for the AVX-512 equivalent implementation in Listing 4.6. A detailed explanation is provided for the AVX-512 equivalent implementation in Listing 4.6.

```
void qd_dd_mul(double a_hi, double a_lo,
               double b_hi, double b_lo,
               double *res_hi, double *res_lo) {
    double c_hi, c_lo;

    c_hi = qd_dd_two_prod(a_hi, b_hi, &c_lo);
    c_lo += (a_hi * b_lo) + (a_lo * b_hi);
    c_hi = qd_dd_quick_two_sum(c_hi, c_lo, &c_lo);

    /* return */ *hi = c_hi; *lo = c_lo;
}
```

Listing 2.5: Double-Double multiplication modified from the QD library [6]. The call to "qd_dd_two_prod" calculates the product of the high precision bits of two double-double numbers. The low precision bits will be arranged from multiply add floating point operations to return a hi and *lo* double-double precision number.

### 2.1.4 Matrix Multiplication

Matrix multiplication has a time complexity of $O(N^3)$, where $N^3$ multiply-add operations are required for an $n \times n$ matrix. This dissertation optimises matrix multiplication algorithms with time complexity $O(N^3)$. Faster matrix multiplication algorithms exist such as Strassen's algorithm with time complexity of approximately $O(N^{2.8074})$, however it is prone to numerical instability [16] and was not chosen because of the instability. Below is the matrix multiplication algorithm written in mathematical notation.

$$c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{in}b_{nj} = \sum_{k=1}^{n} a_{ik}b_{kj}$$

for i = 1, ..., m and j=1, ...,p.

Below is the same algorithm displayed visually where n=3, m=2, and p=1.

$$\overset{3\times2 \text{ matrix}}{\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix}} \overset{2\times1 \text{ matrix}}{\begin{bmatrix} b_{11} \\ b_{21} \end{bmatrix}} = \overset{3\times1 \text{ matrix}}{\begin{bmatrix} c_{11} \\ c_{21} \\ c_{31} \end{bmatrix}}$$

$$\overset{3\times1 \text{ matrix}}{\begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} \\ a_{21}b_{11} + a_{22}b_{21} \\ a_{31}b_{11} + a_{32}b_{21} \end{bmatrix}}$$

The algorithm written in C.

```
void matrix_multiplication(double **a, double **b, double **c,
                           int n, int m, int p) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            for (int k = 0; k < p; k++) {
                c[i][k] += a[i][j] * b[j][k];
            }
        }
    }
}
```

Listing 2.6: Matrix multiplication in C for the double type

#### 2.1.4.1 Cache Blocking/Tiling

Cache blocking or tiling is a well known technique applied to matrix multiplication algorithms. Cache blocking splits a large multiplication into smaller blocks. This reduces the total cache misses and, in turn improves performance [17]. The implementation chapter provides code implementing cache blocking, refer to Listings 4.9, 4.10, 4.11, 4.12, 4.15, and 4.16. The design chapter discusses an algorithm for improving register locality by using AVX-512 registers; see Section 3.3. Intel provides concrete code examples in their website for implementing cache blocking on a general 1-D, 2-D, or 3D spatial data structures [18].

### 2.1.5 Multithreading

Multithreading is the ability of a CPU to provide multiple threads of execution. Matrix multiplication efficiently leverages multithreading by distributing independent tasks across multiple threads of execution.

#### 2.1.5.1 Pthread

POSIX Threads (PThreads) [19] is an Application Programming Interface (API) library which exists independent of programming languages. Pthreads are present in Unix like operating systems and therefore windows operating systems do not support them natively. In C, this library is accessed through the "pthread.h" header. Pthreads are a low level API interface for explicitly managing threads. The API interface enables creating, joining, and using mutexes for enabling threading. This API was not used for this dissertation for its higher complexity and it not being cross platform.

#### 2.1.5.2 OpenMP

OpenMP [20] is an API implemented by compilers as an extension. Both GCC and Clang compilers discussed in this dissertation are supported with OpenMP. In C it is required to include the header "omp.h" and enable the flag "-fopenmp" to enable OpenMP. Listing 2.7 is an example of multithreading in OpenMP. OpenMP is cross platform and is the multithreading API chosen for this dissertation. The multithreaded matrix multiplication algorithms using OpenMP are seen in Listings 4.12, 4.11, and 4.16.

```
void simple(int n, float *a, float *b)
{
    int i;

#pragma omp parallel for
    for (i=1; i<n; i++) /* i is private by default */
        b[i] = (a[i] + a[i-1]) / 2.0;
}
```

Listing 2.7: simple parallel loop in C using OpenMP from code example [20]

# Chapter 3

# Design

This chapter goes over the design of a register locality optimised efficient matrix multiplication algorithm with an emphasis on optimising the use of registers effectively using the features of AVX-512. This chapter covers the abstract design with corresponding code snippets where relevant.

## 3.1 Overview of the Design

The algorithm discussed in this chapter optimises for register locality of matrix multiplication. AVX-512 provides 32 registers each up to 512 bits, the algorithm makes use of reusing registers for loading, storing and performing operations to minimise the requirement of loading from main memory which can be a bottleneck of matrix multiplication algorithms. The algorithm that was determined to be fastest optimising for register locality was Listing 4.16 using optimisations of AVX-512, memory alignment, memory layout, contiguous memory allocation, multithreading, loading additional vector from the matrix and tiling.

## 3.2 Layout of Data

The layout of data can drastically change the performance of algorithms. This section discusses memory layout and how memory allocations can improve performance. The layout of matrices may impact performance. Structure of Arrays and Arrays of Structure data layout is discussed and the optimal layout is chosen based on efficiency; see Sections 3.2.2 and 3.2.3. Examples of two different matrix layouts are presented and the most appropriate layout is chosen based on efficiency. AVX-512 requires memory to be contiguous for it to load from memory to a 512 bit wide vector. Matrix multiplication is as

discussed, row of matrix A by column of matrix B. An approach to ensure memory layout is contiguous is to transpose a matrix. Section 3.2.5 explains why the approach works.

Two structures representing double-double precision numbers can be formed, an array of structures and structure of arrays; see Listings 3.1 and 3.5.

```
typedef struct {
    double hi;
    double lo;
} DoubleDouble;
```

Listing 3.1: Structure for use with array of structures double-double in C, discussed in 3.2 and 3.3.

### 3.2.1 Memory Layout

Two common forms of allocations memory exist for a two dimensional array structure. A 2D array style memory allocation, and a 1D contiguous memory allocation. The 1D memory approach is more efficient as the values are always next to each other and memory indrections using pointers do not exist. An example in Listing 3.2 defines a memory allocation for a 2D structure.

```
DoubleDouble **matrix_2d = malloc(size * sizeof(DoubleDouble*));
for (int i = 0; i < size; i++) {
    matrix[i] = malloc(size * sizeof(DoubleDouble));
}
DoubleDouble value = matrix_2d[i][j];
```

Listing 3.2: 2D memory allocation of a double-double structure in C. Indexing is done using built in array indexing.

The approach of forming one contiguous piece of memory looks is shown in Figure 3.3.

```
DoubleDouble *matrix_1d = malloc(size * size *
                                 sizeof(DoubleDouble));
DoubleDouble value =
    matrix_1d[i * size + j]; // computes matrix_2d[i][j]
```

Listing 3.3: 1D contiguous memory of a double-double structure in C. The contiguous memory is emulated by multiplying the $i$ index by size to compute what would be *matrix_2d[i][j]* from 3.2.

The matrix layout is vital for improving memory locality. In this section, two data layout patterns, a structure of arrays, and an array of structures are discussed. Visual examples are presented with reasoning for why the structure of arrays approach is better than the array of structures for double-double precision matrix multiplication.

Another optimisation helping with reducing cache misses, is aligning memory; see Listing 3.4. In the AVX-512 vector instruction set, an intrinsic by the name of *_mm512_load_pd* exists. When memory is not aligned the intrinsics *_mm512_loadu_pd* is used. The load aligned function is used to load aligned memory, giving additional runtime improvements. AVX-512 requires 64 byte aligned. The listing below demonstrates alignment of memory.

```
DoubleDouble *matrix_1d_aligned =
    _mm_malloc(size * size * sizeof(DoubleDouble), 64);
```

Listing 3.4: Aligning memory using "_mm_malloc" by 64 bytes built in to the AVX-512 vector instruction set in C.

### 3.2.2 Array of Structures

A double-double matrix can be defined using an array of structures, where a structure is just a broad abstraction of a double-double precision number. I refer to a double-double number as *hilo*, where *hi* is the high 64 bits, and *lo* is the low 64 bits. Figure 3.1, represents an array of structures matrix.

$$\overset{8\times 8 \text{ matrix}}{\begin{bmatrix} hilo_{11} & hilo_{12} & hilo_{13} & \cdots & hilo_{18} \\ hilo_{21} & hilo_{22} & hilo_{23} & \cdots & hilo_{28} \\ hilo_{31} & hilo_{32} & hilo_{33} & \cdots & hilo_{38} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ hilo_{81} & hilo_{82} & hilo_{83} & \cdots & hilo_{88} \end{bmatrix}}$$

Figure 3.1: Array of Structure layout layout for a matrix. Corresponding code of array of structures 3.3.

The problem with loading contiguous memory of *hilo* values from the matrix is that interleaving the values into separate registers is a requirement. The functions defined in Chapter 2 for double-double precision arithmetic 2.1.3.3 require two operations of interleaving the data to isolate the *hi* from the *lo*, if SIMD is used to vectorise matrix multiplication. Another issue is the interleaved values have 256 bits instead of 512 bits, so two additional loads followed by interleaving is required to fill in the two registers to have eight *hi* values and eight *lo* values.

$$vector1 = \begin{bmatrix} hi_{11}, & lo_{11}, & hi_{12}, & lo_{12}, & \cdots, & hi_{18}, & lo_{18} \end{bmatrix}$$

$$vector2 = \begin{bmatrix} hi_{11}, & lo_{11}, & hi_{12}, & lo_{12}, & \cdots, & hi_{18}, & lo_{18} \end{bmatrix}$$

$$vector1hi = \begin{bmatrix} hi_{11}, & hi_{12}, & hi_{13}, & hi_{14} \end{bmatrix}$$

$$vector1lo = \begin{bmatrix} lo_{11}, & lo_{12}, & lo_{13}, & lo_{14} \end{bmatrix}$$

### 3.2.3 Structure of Arrays

Structure of Arrays is the data layout that solves the problem of interleaving values into separate registers and loading additional memory. Instead of defining a matrix to have double-double precision numbers this approach defines two matrices, hence the name structure of the arrays. Figure 3.2, represents the structure of arrays matrix where there are *hi* and *lo* matrices.

$$\overset{8\times8 \text{ matrix}}{\begin{bmatrix} hi_{11} & hi_{12} & hi_{13} & \cdots & hi_{18} \\ hi_{21} & hi_{22} & hi_{23} & \cdots & hi_{28} \\ hi_{31} & hi_{32} & hi_{33} & \cdots & hi_{38} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ hi_{81} & hi_{82} & hi_{83} & \cdots & hi_{88} \end{bmatrix}}$$

$$\overset{8\times8 \text{ matrix}}{\begin{bmatrix} lo_{11} & lo_{12} & lo_{13} & \cdots & lo_{18} \\ lo_{21} & lo_{22} & lo_{23} & \cdots & lo_{28} \\ lo_{31} & lo_{32} & lo_{33} & \cdots & lo_{38} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ lo_{81} & lo_{82} & lo_{83} & \cdots & lo_{88} \end{bmatrix}}$$

Figure 3.2: Structure of Arrays layout for a matrix

$$vector1 = \begin{bmatrix} hi_{11}, & hi_{12}, & hi_{13}, & hi_{14}, & hi_{15}, & hi_{16}, & hi_{17}, & hi_{18} \end{bmatrix}$$

$$vector2 = \begin{bmatrix} lo_{11}, & lo_{12}, & lo_{13}, & lo_{14}, & lo_{15}, & lo_{16}, & lo_{17}, & lo_{18} \end{bmatrix}$$

The corresponding structure in C and the memory allocation is shown in Listing 3.5.

```
typedef struct {
    double* hi;
    double* lo;
} DoubleDouble_SOA_FLAT;
```

Listing 3.5: Structure of Arrays of double-double values in C.

The memory allocation of this structure requires two allocations[1]; see Listing 3.6.

```
DoubleDouble_SOA_FLAT *matrix;
matrix->hi = _mm_malloc(n * n * sizeof(double), 64);
matrix->lo = _mm_malloc(n * n * sizeof(double), 64);
```

Listing 3.6: Aligning memory using "_mm_malloc" by 64 bytes built in to the AVX-512 vector instruction set in C.

### 3.2.4 Setting result matrix to zero

Initialising the result matrix to zero is necessary for Listings 4.7, 4.8, 4.9, 4.10, 4.15, and 4.16. The Listings 4.12, 4.13, and 4.14 do not require the result matrix be initialised to zero because they use an accumulator which gets reset. Matrix multiplication tiling is discussed in Chapter 2, it requires the result matrix to be zero because tiling is a memory optimisation that will read the result matrix in its computation. Reference Listing 3.7 for setting the matrix to zero.

```
DoubleDouble_SOA_FLAT result;
/* allocate memory for result */
memset(result.hi, 0.0, size * size);
memset(result.lo, 0.0, size * size);
```

Listing 3.7: Setting result matrix to zero using *memset* after allocating memory in C.

[1] An alternative implementation can improve the allocations to one, by storing an offset value to *lo*.

### 3.2.5 Transposing

Transposing a matrix is defined as flipping the diagonals of a matrix. Transposing a matrix allows AVX-512 to load contiguous values from the columns of the original matrix before transposing. Transposing one of the matrices to load contiguous memory is vital for AVX-512 to load in one instruction rather than eight without contiguous memory. Given a matrix n=8, m=2, and p=8. The matrix multiplication looks like the following:

$$\overset{n\times m \text{ matrix}}{\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \\ a_{41} & a_{42} \\ a_{51} & a_{52} \\ a_{61} & a_{62} \\ a_{71} & a_{72} \\ a_{81} & a_{82} \end{bmatrix}} \overset{m\times p \text{ matrix}}{\begin{bmatrix} b_{11} & b_{12} & b_{13} & b_{14} & b_{15} & b_{16} & b_{17} & b_{18} \\ b_{21} & b_{22} & b_{23} & b_{24} & b_{25} & b_{26} & b_{27} & b_{28} \end{bmatrix}} = \overset{n\times p \text{ matrix}}{\begin{bmatrix} c_{11} & c_{12} & c_{13} & c_{14} & c_{15} & c_{16} & c_{17} & c_{18} \\ c_{21} & c_{22} & c_{23} & c_{24} & c_{25} & c_{26} & c_{27} & c_{28} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ c_{81} & c_{82} & c_{83} & c_{84} & c_{85} & c_{86} & c_{87} & c_{88} \end{bmatrix}}$$

Figure 3.3: Naive matrix multiplication.

In this example, transposing the first matrix results in the first matrix having the same number of rows as the second matrix it is multiplying by. The naive matrix multiplication algorithm no longer works due to the rows becoming columns and columns the rows for the first matrix. The intuition and explanation of what set of operations can be combined to perform a matrix multiplication is explained following this.

$$\overset{m\times n \text{ matrix}}{\begin{bmatrix} a_{11} & a_{21} & a_{31} & a_{41} & a_{51} & a_{61} & a_{71} & a_{81} \\ a_{12} & a_{22} & a_{32} & a_{42} & a_{52} & a_{62} & a_{72} & a_{82} \end{bmatrix}} \overset{m\times p \text{ matrix}}{\begin{bmatrix} b_{11} & b_{12} & b_{13} & b_{14} & b_{15} & b_{16} & b_{17} & b_{18} \\ b_{21} & b_{22} & b_{23} & b_{24} & b_{25} & b_{26} & b_{27} & b_{28} \end{bmatrix}}$$

Figure 3.4: The left matrix transposed, where the number of rows are equivalent to the right matrix.

## 3.3 Intuition of Algorithm

This section discusses the intuition of a different way of performing matrix multiplication. This algorithm optimises the register locality in order to reuse registers and reduce memory loads and stores, and accumulate the result of the double-double multiply-add instructions in registers rather than directly to memory.

$$
\begin{aligned}
c_{11} &= a_{11}b_{11} + a_{12}b_{21} \\
c_{12} &= a_{11}b_{12} + a_{12}b_{22} \\
c_{13} &= a_{11}b_{13} + a_{12}b_{23} \\
c_{14} &= a_{11}b_{14} + a_{12}b_{24} \\
c_{15} &= a_{11}b_{15} + a_{12}b_{25} \\
c_{16} &= a_{11}b_{16} + a_{12}b_{26} \\
c_{17} &= a_{11}b_{17} + a_{12}b_{27} \\
c_{18} &= a_{11}b_{18} + a_{12}b_{28}
\end{aligned}
$$

Figure 3.5: Naive matrix multiplication example with the resulting "c" values from multiply-add operations of the first row of first matrix and all columns of second matrix. Multiple loading operations of the same memory elements in $a_{11}$ and $a_{12}$ occur.

A naive matrix multiplication algorithm as seen in Figure 3.3 would perform the following operations to obtain the first row of the resulting matrix. To obtain the first row, the values $a_{11}$ and $a_{12}$ are reused eight times. In a naive implementation of matrix multiplication this would load these two values from memory eight times. This is what this algorithm optimises by transposing the matrix.

### 3.3.1 Transposing matrix to reduce loads

The transpose of the first matrix can help reduce the loads by the following set of operations.

1. Load the row of the first transposed matrix into a register
2. For each value loaded into the register perform a broadcast shuffle operation or a left shift cyclical rotation. A broadcast shuffle operation in practice allows any value in the eight lane AVX-512 register to be set in all lanes of the register.
3. Perform the double-double multiply-add operation for each broadcast shuffled registers. In AVX-512 this would equate to eight double-double multiply-add operations.
4. Accumulate the result in eight separate registers where each lane of the resulting register corresponds to the multiply-add operations as shown in the naive implementation in Figure 3.5.

### 3.3.2 Broadcast shuffle

Given value loaded from contiguous memory the resulting register of the left matrix. Below I label my vectors with the prefix *zmm*[2].

$$zmm0 = [a_{11}, a_{21}, a_{31}, a_{41}, a_{51}, a_{61}, a_{71}, a_{81}]$$

The broadcast shuffle of each value then gives eight different registers with each lane having the same value. This operation is inexpensive as compared to loading from memory.

$$zmm1 = [a_{11}, a_{11}, a_{11}, a_{11}, a_{11}, a_{11}, a_{11}, a_{11}]$$
$$zmm2 = [a_{21}, a_{21}, a_{21}, a_{21}, a_{21}, a_{21}, a_{21}, a_{21}]$$
$$zmm3 = [a_{31}, a_{31}, a_{31}, a_{31}, a_{31}, a_{31}, a_{31}, a_{31}]$$
$$zmm4 = [a_{41}, a_{41}, a_{41}, a_{41}, a_{41}, a_{41}, a_{41}, a_{41}]$$
$$zmm5 = [a_{51}, a_{51}, a_{51}, a_{51}, a_{51}, a_{51}, a_{51}, a_{51}]$$
$$zmm6 = [a_{61}, a_{61}, a_{61}, a_{61}, a_{61}, a_{61}, a_{61}, a_{61}]$$
$$zmm7 = [a_{71}, a_{71}, a_{71}, a_{71}, a_{71}, a_{71}, a_{71}, a_{71}]$$
$$zmm8 = [a_{81}, a_{81}, a_{81}, a_{81}, a_{81}, a_{81}, a_{81}, a_{81}]$$

The equivalent C code for performing a broadcast shuffle for some memory loaded into a vector is presented in Listing 3.8. The "__mm512_set1_pd" intrinsic performs the broadcast shuffle in AVX-512.

```
__m512d a8_hi = _mm512_load_pd(&a->hi[k*n + i]);
__m512d a8_lo = _mm512_load_pd(&a->lo[k*n + i]);
/* load b vectors for hi and lo */
for (int l = 0; l < 8; l++) {
    __m512d a8_hi_l = _mm512_set1_pd(a8_hi[l]);
    __m512d a8_lo_l = _mm512_set1_pd(a8_lo[l]);
    /* Perform double-double multiply-add operations */
}
```

Listing 3.8: An example of performing a broadcast shuffle eight times on AVX-512 vectors for the *hi* and *lo* values of a double-double precision matrix.

[2] *zmm* in x86/x64 assembly refer to registers. This chapter refers to *zmm* as variable names not registers. The compiler determines which registers between *zmm0-zmm31* are assigned.

### 3.3.3 Left Shift Cyclical Rotation

Another approach as mentioned in the footnote is to shift values to the left in a cyclical pattern for all lanes in the vector minus one. For AVX-512 that would be seven rotations. The first vector can be reused as a 0 rotation vector. An example of a vector with values is the following:

$$zmm0 = [a_{11}, a_{21}, a_{31}, a_{41}, a_{51}, a_{61}, a_{71}, a_{81}]$$

The result of each left shift cyclical rotation results in eight register accumulators.

$$zmm1 = [a_{21}, a_{31}, a_{41}, a_{51}, a_{61}, a_{71}, a_{81}, a_{11}]$$
$$zmm2 = [a_{31}, a_{41}, a_{51}, a_{61}, a_{71}, a_{81}, a_{11}, a_{21}]$$
$$zmm3 = [a_{41}, a_{51}, a_{61}, a_{71}, a_{81}, a_{11}, a_{21}, a_{31}]$$
$$zmm4 = [a_{51}, a_{61}, a_{71}, a_{81}, a_{11}, a_{21}, a_{31}, a_{41}]$$
$$zmm5 = [a_{61}, a_{71}, a_{81}, a_{11}, a_{21}, a_{31}, a_{41}, a_{51}]$$
$$zmm6 = [a_{71}, a_{81}, a_{11}, a_{21}, a_{31}, a_{41}, a_{51}, a_{61}]$$
$$zmm7 = [a_{81}, a_{11}, a_{21}, a_{31}, a_{41}, a_{51}, a_{61}, a_{71}]$$

Listing 3.9 presents the code for performing a left shift cyclical rotation. The AVX-512 vector instruction set does not support a left shift cyclical rotation, instead it supports a right shift cyclical rotation. The instruction "__mm512_alignr_epi64" is used to implement a right shift cyclical rotation. Any right cyclical rotation can be described in terms of left cyclical rotation. The "__mm512_alignr_epi64" instruction only works on 64 bit integers in a 512 bit wide vector. AVX-512 was created in way where there are different data types referring to the same piece memory of memory, so functions which take integer values must first be cast to integers using "__mm512_castpd_si512" then cast back to doubles using "__mm512_castsi512_pd". The reason this works is because inherently 64 bit integers takes the same amount as 64 bit doubles, so any shifting operation will work identical on any data type. Type casting is a requirement by compiler intrinsics for the AVX-512 vector instruction set. Type casting is zero cost, meaning the compiled code will not perform type casting. The compiled binary code does not have a notion of data types and works directly with 512 bits. In this way AVX-512 instructions are considered "weakly typed". The data types can be cast to any other data type without the compiler warnings or errors.

```
__m512d zmm0 = _mm512_load_pd(&array[index]);
__m512d zmm1 = _mm512_castsi512_pd(_mm512_alignr_epi64(
               _mm512_castpd_si512(zmm0),
               _mm512_castpd_si512(zmm0), 7));
__m512d zmm2 = _mm512_castsi512_pd(_mm512_alignr_epi64(
               _mm512_castpd_si512(zmm0),
               _mm512_castpd_si512(zmm0), 6));
__m512d zmm3 = _mm512_castsi512_pd(_mm512_alignr_epi64(
               _mm512_castpd_si512(zmm0),
               _mm512_castpd_si512(zmm0), 5));
__m512d zmm4 = _mm512_castsi512_pd(_mm512_alignr_epi64(
               _mm512_castpd_si512(zmm0),
               _mm512_castpd_si512(zmm0), 4));
__m512d zmm5 = _mm512_castsi512_pd(_mm512_alignr_epi64(
               _mm512_castpd_si512(zmm0),
               _mm512_castpd_si512(zmm0), 3));
__m512d zmm6 = _mm512_castsi512_pd(_mm512_alignr_epi64(
               _mm512_castpd_si512(zmm0),
               _mm512_castpd_si512(zmm0), 2));
__m512d zmm7 = _mm512_castsi512_pd(_mm512_alignr_epi64(
               _mm512_castpd_si512(zmm0),
               _mm512_castpd_si512(zmm0), 1));
```

Listing 3.9: AVX-512 code snippet for performing a left shift cyclical rotation eight times from a loaded vector.

The 8x8 accumulator requires 28 vertical swaps between register values. The performance impact is negligible because the operation is performed after computing the entire 8x8 accumulator. As an example, 28 swaps are performed after computing double-double multiply-add operation for a 1024x1024 matrix. Below is a list of vectors with their swaps labelled for the elements $z_{i,j}$.

$$zmm0 = [z_{0,0}, z_{0,1}, z_{0,2}, z_{0,3}, z_{0,4}, z_{0,5}, z_{0,6}, z_{0,7}]$$
$$zmm1 = [z_{1,0}, z_{1,1}, z_{1,2}, z_{1,3}, z_{1,4}, z_{1,5}, z_{1,6}, z_{1,7}]$$
$$zmm2 = [z_{2,0}, z_{2,1}, z_{2,2}, z_{2,3}, z_{2,4}, z_{2,5}, z_{2,6}, z_{2,7}]$$
$$zmm3 = [z_{3,0}, z_{3,1}, z_{3,2}, z_{3,3}, z_{3,4}, z_{3,5}, z_{3,6}, z_{3,7}]$$
$$zmm4 = [z_{4,0}, z_{4,1}, z_{4,2}, z_{4,3}, z_{4,4}, z_{4,5}, z_{4,6}, z_{4,7}]$$
$$zmm5 = [z_{5,0}, z_{5,1}, z_{5,2}, z_{5,3}, z_{5,4}, z_{5,5}, z_{5,6}, z_{5,7}]$$
$$zmm6 = [z_{6,0}, z_{6,1}, z_{6,2}, z_{6,3}, z_{6,4}, z_{6,5}, z_{6,6}, z_{6,7}]$$
$$zmm7 = [z_{7,0}, z_{7,1}, z_{7,2}, z_{7,3}, z_{7,4}, z_{7,5}, z_{7,6}, z_{7,7}]$$

The swaps are performed as follows:

Swap Column 0: $z_{0,0} \leftrightarrow z_{7,0}, z_{1,0} \leftrightarrow z_{6,0}, z_{2,0} \leftrightarrow z_{5,0}, z_{3,0} \leftrightarrow z_{4,0}$
Swap Column 1: $z_{1,1} \leftrightarrow z_{7,1}, z_{2,1} \leftrightarrow z_{6,1}, z_{3,1} \leftrightarrow z_{5,1}$
Swap Column 2: $z_{0,2} \leftrightarrow z_{1,2}, z_{2,2} \leftrightarrow z_{7,2}, z_{3,2} \leftrightarrow z_{6,2}, z_{4,2} \leftrightarrow z_{5,2}$
Swap Column 3: $z_{0,3} \leftrightarrow z_{2,3}, z_{3,3} \leftrightarrow z_{7,3}, z_{4,3} \leftrightarrow z_{6,3}$
Swap Column 4: $z_{0,4} \leftrightarrow z_{3,4}, z_{1,4} \leftrightarrow z_{2,4}, z_{4,4} \leftrightarrow z_{7,4}, z_{5,4} \leftrightarrow z_{6,4}$
Swap Column 5: $z_{0,5} \leftrightarrow z_{4,5}, z_{1,5} \leftrightarrow z_{3,5}, z_{5,5} \leftrightarrow z_{7,5}$
Swap Column 6: $z_{0,6} \leftrightarrow z_{5,6}, z_{1,6} \leftrightarrow z_{4,6}, z_{2,6} \leftrightarrow z_{3,6}, z_{6,6} \leftrightarrow z_{7,6}$
Swap Column 7: $z_{0,7} \leftrightarrow z_{6,7}, z_{1,7} \leftrightarrow z_{5,7}, z_{2,7} \leftrightarrow z_{4,7}$

Figure 3.6: The 28 vertical swaps illustrated using the left cyclical approach. Accessing the individual values of the lanes to requires permutation of registers. The total amount of swaps is negligible due to the swaps occurring after the accumulators have computed the double-double multiply-add operations for the total number of rows in the matrix on on set of column values.

Listing 3.10 presents a code snippet of swapping the first column as described in Figure 3.6. The following swap operations must be repeated for the remaining seven columns of the 8x8 accumulator.

```
double temp;
temp = zmm0[0];
zmm0[0] = zmm7[0];
zmm7[0] = temp;

temp = zmm1[0];
zmm1[0] = zmm6[0];
zmm6[0] = temp;

temp = zmm2[0];
zmm2[0] = zmm5[0];
zmm5[0] = temp;

temp = zmm3[0];
zmm3[0] = zmm4[0];
zmm4[0] = temp;
```

Listing 3.10: AVX-512 code snippet for performing a left shift cyclical rotation eight times from a loaded vector.

The left shift cyclical rotation may not be the only sequence of permutations which result in a correct solution. There may be different more efficient permutations, this is one simple implementation of what works. Different permutations would result in a different number of swaps. The left shift cyclical rotation must be executed for *hi* and *lo* vectors totalling 56 vertical swaps.

### 3.3.4 Multiply-Add Operation

Performing the multiply-add operation after the broadcast shuffle results in the following sequence, where the following operation described below is repeated for the remaining seven vectors of the broadcast shuffle[3]. The following operation is then repeated for the remaining rows of the left and right matrix. In this example I am referencing 3.4.

$$zmm0 = [a_{11}, a_{11}, a_{11}, a_{11}, a_{11}, a_{11}, a_{11}, a_{11}]$$

$$\times$$

$$zmm1 = [b_{11}, b_{12}, b_{13}, b_{14}, b_{15}, b_{16}, b_{17}, b_{18}]$$

$$=$$

$$zmm2 = [a_{11}b_{11}, a_{11}b_{12}, a_{11}b_{12}, a_{11}b_{13}, a_{11}b_{14}, a_{11}b_{15}, a_{11}b_{16}, a_{11}b_{17}, a_{11}b_{18}]$$

*zmm2* is the accumulator vector which accumulates the addition of the multiply operations. For the remaining rows of the matrix, the following operation occurs after the broadcast shuffle of row 2 of the left and right matrix if only considering the first vector after the broadcast shuffle. The remaining seven vectors from the broadcast shuffle follows the same operation. The seven remaining vectors are the seven accumulators of the resulting matrix. So the first vector of the broadcast shuffle only results in the first row of the resulting matrix. The implications mean, that the algorithm optimises and always produces an 8x8 result rather than a naive matrix multiplication algorithm which produces a 1x1 result since it does not use SIMD.

$$zmm0 = [a_{12}, a_{12}, a_{12}, a_{12}, a_{12}, a_{12}, a_{12}, a_{12}]$$

$$\times$$

$$zmm1 = [b_{21}, b_{22}, b_{23}, b_{24}, b_{25}, b_{26}, b_{27}, b_{28}]$$

$$+ =$$

$$
\begin{aligned}
zmm2 = [&a_{11}b_{11} + a_{12}b_{21},\\
&a_{11}b_{12} + a_{12}b_{22},\\
&a_{11}b_{12} + a_{12}b_{22},\\
&a_{11}b_{13} + a_{12}b_{23},\\
&a_{11}b_{14} + a_{12}b_{24},\\
&a_{11}b_{15} + a_{12}b_{25},\\
&a_{11}b_{16} + a_{12}b_{26},\\
&a_{11}b_{17} + a_{12}b_{27},\\
&a_{11}b_{18} + a_{12}b_{28}]
\end{aligned}
$$

The register accumulator approach is identical to the naive matrix multiplication approach; reference Figure 3.5 for the comparison.

### 3.3.5 Using an additional row from the right matrix

The algorithm as described above can be extended. The operations are near identical, but the matrix on the right can have two rows loaded. Consider the following matrices where the left matrix was transposed utilising the same broadcast shuffle or left shift cyclical rotation approach.

$$\overset{m\times n \text{ matrix}}{\begin{bmatrix} a_{11} & a_{21} & a_{31} & a_{41} & a_{51} & a_{61} & a_{71} & a_{81} \\ a_{12} & a_{22} & a_{32} & a_{42} & a_{52} & a_{62} & a_{72} & a_{82} \end{bmatrix}} \overset{m\times p \text{ matrix}}{\begin{bmatrix} b_{11} & b_{12} & b_{13} & b_{14} & b_{15} & b_{16} & b_{17} & b_{18} & \cdots & b_{126} \\ b_{21} & b_{22} & b_{23} & b_{24} & b_{25} & b_{26} & b_{27} & b_{28} & \cdots & b_{226} \end{bmatrix}}$$

$$zmm0 = [a_{12}, a_{12}, a_{12}, a_{12}, a_{12}, a_{12}, a_{12}, a_{12}]$$

$$\times$$

$$zmm1 = [b_{21}, b_{22}, b_{23}, b_{24}, b_{25}, b_{26}, b_{27}, b_{28}]$$

$$=$$

$$zmm3$$

$$zmm0 = [a_{12}, a_{12}, a_{12}, a_{12}, a_{12}, a_{12}, a_{12}, a_{12}]$$

$$\times$$

$$zmm2 = [b_{220}, b_{221}, b_{222}, b_{223}, b_{224}, b_{225}, b_{226}, b_{227}]$$

$$=$$

$$zmm4$$

The approach of loading two rows of the right matrix reduces the total amount of loads required from the left matrix, because the values loaded from the left matrix are being reused for two separate double-double 8x8 accumulators.

### 3.3.6 High Register Usage

This section discusses where in the design are the registers used. In the broadcast shuffle implementation, the row of the left matrix is used and eight vectors for each broadcast shuffle. The 16 values from the right matrices rows are used which results in two registers, the accumulators require 16 registers, two sets of eight registers which results in an 8x8 accumulator, as shown in Section 3.3.3. Double-Double floating arithmetic operations requires additional registers for the 28 floating point arithmetic instructions in AVX-512, as seen in Listings 4.1, 4.2, 4.3, 4.4, and 4.6. The data layout of using structure of arrays approach results in one load from *hi* matrix, and one load for the *lo* matrix. In general many of the total 32 registers in AVX-512 are being utilised. The compiler deals with many of the data dependency issues with reusing registers by reassigning used registers so it is not clear the total registers that are used for computing a matrix multiplication but it is most likely close to 32 registers considering all the registers being used for this solution.

## 3.4 Summary

In summary, the design of the register locality optimised matrix multiplication was described in detail with reference to the registers and vector operations in the AVX-512 vector instruction set. The data and memory layout were detailed and the intuition of the algorithm was presented with detailed examples for how the algorithm works in a high level description.

[3]The left shift cyclical rotation approach performs the same multiply-add operations with the difference being the requirement of the 28 swaps for *hi* and *lo* after performing all multiply-add operations for the number of rows in the left matrix and right matrix.

# Chapter 4

# Implementation

This chapter presents the AVX-512 code for performing double-double operations in Listings 4.1, 4.2, 4.3, 4.4, 4.6. Double-Double matrix multiplication in Listings 4.7, 4.8, and 4.9 do not use the AVX-512 vector instruction set. The listings that do not utilise AVX-512 were created to compare against their AVX-512 counterparts in Chapter 5.

## 4.1 Double-Double AVX-512 modifications

I referenced the addition and multiplication functions from the QD library [6] in Chapter 2 for the AVX-512 implementations which were created specifically for this dissertation. The Listings 4.1, 4.2, 4.3, 4.4, and 4.6 enable double-double precision arithmetic operations across 512 bit wide AVX-512 vectors. The list of AVX-512 instructions required to implement double-double precision operations on AVX-512 are the following:

- __mm512_add_pd - Performs an addition instruction between two 512 bit wide vectors, which hold eight double values each.
- __mm512_sub_pd - Performs an subtraction instruction between two 512 bit wide vectors, which hold eight double values each.
- __mm512_mul_pd - Performs a multiplication instruction between two 512 bit wide vectors, which hold eight double values each.
- __mm512_fmadd_pd - Performs a fused multiply-add instruction on three 512 bit wide vectors, where the first two parameters perform addition, and the last parameter performs the multiplication.
- __mm512_store_pd - Performs a 64 byte aligned stores instruction to the memory a memory location storing in the 512 bit wide vector into the memory address.

The AVX-512 instructions are required to replace the existing scalar instructions $(+, -, *)$ which do not perform arithmetic instructions on vectors from Listings 2.1, 2.2, 2.3, 2.4, and 2.5. The helper functions in Listings 4.1, 4.2, and 4.3 perform addition and multiplication and correctly assign the corresponding *hi* and *lo* value of a double-double precision number ensuring *lo* has the next digits which does not fit into *hi*. The helper functions have defined variable names in a specific way. The main idea is that each of the helper functions returns a result and an error. If two *hi* values are passed then the return values are *hi* and *lo*. If two *lo* values are passed then the return values are *hi_err* and *lo_err*. The *lo_err* signifies that it is an error of an error. The variable names can interchangeably be replaced with *result* and *error* where each iteration of error is the digits which cannot fit into *result*, but the decision to name them after double-double terminology was chosen.

Listing 4.1 is a helper function computing the addition of double 512 bit wide vectors and the corresponding error. This function requires that all values of *a8* are greater or equal to *b8*. The function is used to normalise two doubles into one double-double precision number in Listings 4.4 and 4.6. As an example, if a *hi* and a *lo* value are passed, *hi* takes 64 bits and *lo* takes the next digits of the 128 bit result from addition to result in a double-double precision number.

```
static inline __m512d
qd_dd_quick_two_sum_avx_512(__m512d a8, __m512d b8,
                            __m512d *err8) {
    __m512d s8 = _mm512_add_pd(a8, b8);
    *err8 = _mm512_sub_pd(b8, _mm512_sub_pd(s8, a8));
    return s8;
}
```

Listing 4.1: C code of Quick-Two-Sum using AVX-512.

Listing 4.2 is a helper function computing addition of two double 512 bit wide vectors. This listing does not require the double values of the first vector to be greater or equal to the second vector. This summation is used in Listing 4.4, for performing the addition between two *lo* values of a double-double precision number and getting the result which fits into the *lo* result, but the error does not and is discarded.

```
static inline __m512d
qd_dd_two_sum_avx_512(__m512d a8, __m512d b8,
                      __m512d *err8) {
    __m512d s8 = _mm512_add_pd(a8, b8);
    __m512d bb8 = _mm512_sub_pd(s8, a8);
    *err8 = _mm512_add_pd(
                _mm512_sub_pd(a8, _mm512_sub_pd(s8, bb8)),
                _mm512_sub_pd(b8, bb8));
    return s8;
}
```

Listing 4.2: C code of Quick-Two-Sum using the AVX-512 instruction set.

Listing 4.3 is a helper function computing the multiplication of two double 512 bit wide vectors. A fused multiply-add instruction calculates the error term of multiplication between two double values. This function is used in Listing 4.6 for computing double-double multiplication. As an example passing two *hi* values into this function computes the *hi* and *lo* values of the resulting *hi* values from multiplication.

```
static inline __m512d
qd_dd_two_prod_avx_512(__m512d a, __m512d b, __m512d *err) {
    __m512d p8 = _mm512_mul_pd(a, b);
    *err = _mm512_fmsub_pd(a, b, p8);
    return p8;
}
```

Listing 4.3: C code of Two-Prod using the AVX-512 instruction set.

Listing 4.4 computes the addition between double-double floating precision numbers of 512 bit wide vectors. The double-double addition satisfies the IEEE style error bound of floating point numbers. This means the resulting double-double number ensures to have an error between the *hi* and *lo* of no more than the IEEE specification agrees upon for double precision numbers. Listings 4.1 and 4.2 are used to correctly perform addition of double-double numbers. The Listings 4.10 and 4.11 inline an AVX-512 store instruction after addition instead of accumulating the result.

```
static inline void
qd_dd_ieee_add_avx_512(__m512d a_hi, __m512d a_lo,
                       __m512d b_hi, __m512d b_lo,
                       double *c_hi, double *c_lo) {
    /* This one satisfies IEEE style error bound,
       due to K. Briggs and W. Kahan.                    */
    __m512d c_hi_8, c_lo_8, c_hi_err_8, c_lo_err_8;

    c_hi_8 = qd_dd_two_sum_avx_512(a_hi, b_hi, &c_lo_8);
    c_hi_err_8 = qd_dd_two_sum_avx_512(a_lo, b_lo, &c_lo_err_8);
    c_lo_8 = _mm512_add_pd(c_lo_8, c_hi_err_8);
    c_hi_8 = qd_dd_quick_two_sum_avx_512(c_hi_8, c_lo_8,
                                         &c_lo_8);
    c_lo_8 = _mm512_add_pd(c_lo_8, c_lo_err_8);
    c_hi_8 = qd_dd_quick_two_sum_avx_512(c_hi_8, c_lo_8,
                                         &c_lo_8);

    _mm512_store_pd(c_hi, c_hi_8);
    _mm512_store_pd(c_lo, c_lo_8);
}
```

Listing 4.4: C code of "qd-dd-ieee-add" using the AVX-512 instruction set. A store is performed after the result is calculated. This code segment is used in Listings 4.10 and 4.11.

Listing 4.5 performs the addition in the same way Listing 4.4 performs it for 512 bit wide vectors, but it returns a pointer to a vector and does not store to memory. This function is used in Listings 4.12, 4.14, 4.15, and 4.16 to accumulate the addition rather than storing directly to memory after each double-double multiply-add is calculated.

```
static inline void
qd_dd_ieee_add_avx_512_no_store(__m512d a_hi, __m512d a_lo,
                        __m512d b_hi, __m512d b_lo,
                        __m512d *c_hi, __m512d *c_lo) {
    /* This one satisfies IEEE style error bound,
       due to K. Briggs and W. Kahan.                    */
    __m512d c_hi_8, c_lo_8, c_hi_err_8, c_lo_err_8;

    c_hi_8 = qd_dd_two_sum_avx_512(a_hi, b_hi, &c_lo_8);
    c_hi_err_8 = qd_dd_two_sum_avx_512(a_lo, b_lo, &c_lo_err_8);
    c_lo_8 = _mm512_add_pd(c_lo_8, c_hi_err_8);
    c_hi_8 = qd_dd_quick_two_sum_avx_512(c_hi_8, c_lo_8,
                                         &c_lo_8);
    c_lo_8 = _mm512_add_pd(c_lo_8, c_lo_err_8);
    c_hi_8 = qd_dd_quick_two_sum_avx_512(c_hi_8, c_lo_8,
                                         &c_lo_8);

    /* return */ *c_hi = c_hi_8; *c_lo = c_lo_8;
}
```

Listing 4.5: C code of Quick-Two-Sum using the AVX-512 instruction set. A store is performed after the result is calculated. This code segment is used in matrix multiplication Listings 4.12, 4.14, 4.15, and 4.16.

Listing 4.6 performs multiplication of double-double precision 512 bit wide vectors. The helper function from Listing 4.3 performs multiplication and afterwords the *hi* and *lo* result values are normalised so the *lo* has the next bits which cannot fit in *hi*. The reason the listing does not contain the string "ieee" in its name is because there is no alternative implementation of double-double precision multiplication from the QD library [6], unlike addition which has a "sloppy add" that does not conform to a different style error bound. Only IEEE style error bounded operations were used for this dissertation and the naming of the listings are conforming to the original names in the QD library.

```
static inline void
qd_dd_mul_avx_512(__m512d a_hi, __m512d a_lo,
                  __m512d b_hi, __m512d b_lo,
                  __m512d *c_hi, __m512d *c_lo) {
    __m512d c_hi_8, c_lo_8;

    c_hi_8 = qd_dd_two_prod_avx_512(a_hi, b_hi, &c_lo_8);
    c_lo_8 = _mm512_add_pd(c_lo_8,
                _mm512_fmadd_pd(a_hi, b_lo,
                _mm512_mul_pd(a_lo, b_hi)));
    c_hi_8 = qd_dd_quick_two_sum_avx_512(c_hi_8, c_lo_8,
                                         &c_lo_8);

    /* return */ *c_hi = c_hi_8; *c_lo = c_lo_8;
}
```

Listing 4.6: C equivalent code of qd_mul from the QD library [6]. Source code in Listing 2.5. This code segment is used in matrix multiplication Listings 4.12, 4.10, 4.11, 4.14, 4.15, and 4.16.

## 4.2 Matrix Multiplication Implementations

This section describes the code for matrix multiplication with different optimisations. Each code segment will has a short description. Each listing in this section is prefixed with "gemm", it stands for general matrix multiplication. To reiterate, all listings and optimisations for this dissertation are optimising within the time complexity of $O(N^3)$; see Section 2.1.4. Matrix multiplication implementations from Listings 4.8, 4.9, 4.10, and 4.11 were adjusted from source [21] to use double-double precision, tiling, and multithreading; see Section 2.1.4.1. Matrix multiplication implementations from Listings 4.7, 4.12, 4.13,

4.14, and 4.16 were specifically written for this dissertation without prior matrix multiplication implementation existing beforehand. The Listings 4.12, 4.13, 4.14, 4.15, and 4.16 optimise for register locality based on the discussion of intuition from Section 3.3. The Listings 4.11 and 4.16 were fast using optimisations discussed from Chapter 3.

### 4.2.1 Naive Matrix Multiplication

Listing 4.7 is a naive double-double matrix multiplication without the use of the AVX-512 vector instruction set. This matrix multiplication implementation written for the dissertation has no optimisations. The algorithm is a rewrite of the naive implementation from Listing 2.6 using double-double precision arithmetic operations from the QD library [6] instead of double precision. The purpose of this listing is to compare against optimised matrix multiplication implementations in Chapter 5.

```
void gemm(DoubleDouble_SOA_FLAT *a,
          DoubleDouble_SOA_FLAT *b,
          DoubleDouble_SOA_FLAT *c,
          int n) {
    double res_hi, res_lo;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            for (int k = 0; k < n; k++) {
                qd_dd_mul(a->hi[i*n + k], a->lo[i*n + k],
                          b->hi[k*n + j], b->lo[k*n + j],
                          &res_hi, &res_lo);
                qd_dd_ieee_add(c->hi[i*n + j], c->lo[i*n + j],
                               res_hi, res_lo,
                               &c->hi[i*n + j], &c->lo[i*n + j]);
            }
        }
    }
}
```

Listing 4.7: Naive matrix multiplication algorithm with no optimisations using double-double precision operations from the QD library [6].

### 4.2.2 J-Major Matrix Multiplication

Listing 4.8 is a modification to the naive double-double matrix multiplication from listing 4.7. This modification changes the how the memory is accessed by swapping the $k$ and $j$ loops and making $j$ the most nested loop, the function is j-major because of this. The stride is minimised by accessing contiguous memory from matrix "b" and matrix "c". The values of matrix "a" stay unchanged until exiting the inner loop. This optimisation is a memory access optimisation reducing cache misses by having predictable access patterns.

```
void gemm_j_major(DoubleDouble_SOA_FLAT *a,
                  DoubleDouble_SOA_FLAT *b,
                  DoubleDouble_SOA_FLAT *c,
                  int n) {
    double res_hi, res_lo;
    for (int i = 0; i < n; i++) {
        for (int k = 0; k < n; k++) {
            for (int j = 0; j < n; j++) {
                qd_dd_mul(a->hi[i*n + k], a->lo[i*n + k],
                          b->hi[k*n + j], b->lo[k*n + j],
                          &res_hi, &res_lo);
                qd_dd_ieee_add(c->hi[i*n + j], c->lo[i*n + j],
                               res_hi, res_lo,
                               &c->hi[i*n + j], &c->lo[i*n + j]);
            }
        }
    }
}
```

Listing 4.8: Naive matrix multiplication algorithm modified to access memory more efficiently and use double-double matrix multiplication using QD's library for double-double arithmetic functions as described in the State of the Art chapter [6]. The AVX-512 vector instruction set is not used in this example.

### 4.2.3 J-Major Tiled Matrix Multiplication

Listing 4.9 is a modification of Listing 4.8, where tiling is implemented, reference tiling in Section 2.1.4.1. The listing does not use the AVX-512 vector instruction set but lends itself well for AVX-512 and the following Listing 4.10 utilises AVX-512. Tiling is used to optimise and minimise cache misses by defining a block size of 64. Similarly to Listing

4.8, the code taken from source [21]. This dissertation modified the following code to use double-double precision operations and tiling.

```
static inline
void outer_product(double s_hi, double s_lo,
                   double *b_hi, double *b_lo,
                   double *c_hi, double *c_lo,
                   int n) {
    double res_hi, res_lo;
    for (int j = 0; j < n; j++) {
        qd_dd_mul(s_hi, s_lo, b_hi[j], b_lo[j],
                  &res_hi, &res_lo);
        qd_dd_ieee_add(c_hi[j], c_lo[j], res_hi, res_lo,
                       &c_hi[j], &c_lo[j]);
    }
}
void gemm_j_major_tiling(DoubleDouble_SOA_FLAT *a,
                         DoubleDouble_SOA_FLAT *b,
                         DoubleDouble_SOA_FLAT *c,
                         int n) {
    const int block_size = 64;

    /* Tiling optimisation: Two for loops iterating in blocks */
    for (int ii = 0; ii < n; ii += block_size) {
        for (int kk = 0; kk < n; kk += block_size) {

            /* For loops use indices ii and kk to
             * index to sections (blocks) matrices */
            for (int i = ii; i < ii + block_size
                                && i < n; i++) {
                for (int k = kk; k < kk + block_size
                                && k < n; k++) {
                    outer_product(a->hi[i*n + k], a->lo[i*n + k],
                            &b->hi[k*n], &b->lo[k*n],
                            &c->hi[i* n], &c->lo[i* n], n);
                }
            }
        }
```

```
    }
}
```

Listing 4.9: Matrix multiplication with tiling modified to use double-double operations from the source [21].

### 4.2.4 J-Major Tiled AVX-512 Matrix Multiplication

Listing 4.10 is a modification of Listing 4.9. The matrix multiplication algorithm is taken from source [21]. The function that was taken uses the AVX-512 vector instruction set, but does not implement the double-double operations or tiling. This dissertation implements double-double operations and tiling for Listing 4.10. Section 4.1 provides the relevant listings created for this dissertation to use double-double precision operations with the AVX-512 vector instructions set.

```
static inline
void outer_product_avx_512(__m512d hi, __m512d lo,
                           double *b_hi, double *b_lo,
                           double *c_hi, double *c_lo,
                           int n) {
    __m512d mul_hi, mul_lo;
    for (int j = 0; j < n; j+=8) {
        __m512d b8_hi = _mm512_load_pd(&b_hi[j]);
        __m512d b8_lo = _mm512_load_pd(&b_lo[j]);

        __m512d c8_hi = _mm512_load_pd(&c_hi[j]);
        __m512d c8_lo = _mm512_load_pd(&c_lo[j]);

        qd_dd_mul_avx_512(hi, lo, b8_hi, b8_lo,
                          &mul_hi, &mul_lo);
        qd_dd_ieee_add_avx_512(c8_hi, c8_lo, mul_hi, mul_lo,
                               &c_hi[j], &c_lo[j]);
    }
}
void gemm_j_major_avx_512_tiling(DoubleDouble_SOA_FLAT *a,
                                 DoubleDouble_SOA_FLAT *b,
                                 DoubleDouble_SOA_FLAT *c,
                                 int n) {
```

```
    const int block_size = 64;

    /* Tiling optimisation; Two for loops iterating in blocks */
    for (int ii = 0; ii < n; ii += block_size) {
        for (int kk = 0; kk < n; kk += block_size) {

            /* For loops use indices ii and kk to
             * index to sections (blocks) of matrices */
            for (int i = ii; i < ii + block_size && i < n; i++) {
                for (int k = kk; k < kk + block_size &&
                                 k < n; k+=8) {
                    __m512d va_hi =
                        _mm512_load_pd(&a->hi[i*n + k]);
                    __m512d va_lo =
                        _mm512_load_pd(&a->lo[i*n + k]);
                    for (int l = 0; l < 8; l++) {
                        foo2_avx_512_other(
                                _mm512_set1_pd(va_hi[l]),
                                _mm512_set1_pd(va_lo[l]),
                                &b->hi[(k+l)*n], &b->lo[(k+l)*n],
                                &c->hi[i*n], &c->lo[i*n], n);
                    }
                }
            }
        }
    }
}
```

Listing 4.10: Matrix multiplication algorithm taken from source [21]. The code was modified to use the double-double precision operations. This required AVX-512 double precision operations be replaced with double-double precision operations as presented in Listings 4.1, 4.2, 4.3, 4.4, and 4.6. Tiling was implemented for this dissertation because that optimisation was not present in the original algorithm.

### 4.2.5 J-Major Multithreaded Tiled AVX-512 Matrix Multiplication

Listing 4.11 modifies Listing 4.10 to include multithreading; see Section 2.1.5. The outer most loop is multithreaded. The "c" matrix values must be initialised to zero before calling this function.

```
void gemm_j_major_avx_512_tiling_threaded(
        DoubleDouble_SOA_FLAT *a,
        DoubleDouble_SOA_FLAT *b,
        DoubleDouble_SOA_FLAT *c,
        int n) {
    const int block_size = 64;
    /* Tiling over block_size */
    #pragma omp parallel for /* Multithreading outer loop */
    for (int ii = 0; ii < n; ii += block_size) {
        for (int kk = 0; kk < n; kk += block_size) {
            int i_cond = (ii + block_size < n) ?
                         (ii + block_size) : n;
            int k_cond = (kk + block_size < n) ?
                         (kk + block_size) : n;

            /* Looping and indexing matrix from
             * the ii and kk values */
            for (int i = ii; i < i_cond; i++) {
                for (int k = kk; k < k_cond; k+=8) {
                    __m512d va_hi =
                        _mm512_load_pd(&a->hi[i*n + k]);
                    __m512d va_lo =
                        _mm512_load_pd(&a->lo[i*n + k]);
                    for (int l = 0; l < 8; l++) {
                        outer_product_avx_512(
                                _mm512_set1_pd(va_hi[l]),
                                _mm512_set1_pd(va_lo[l]),
                                &b->hi[(k+l)*n], &b->lo[(k+l)*n],
                                &c->hi[i*n], &c->lo[i*n], n);
                    }
                }
```

```
            }
        }
    }
}
```

Listing 4.11: Multithreaded matrix multiplication using the same algorithm as Listing 4.10 taken from the source [21]. The "c" matrix values must be initialised to zero before calling this function.

### 4.2.6 Left Shift Cyclical Rotation Multithreaded Tiled AVX-512 Matrix Multiplication

Listing 4.12 is the left shift cyclical rotation matrix multiplication algorithm optimising for register locality as described in Section 3.3. All register locality optimised algorithms require matrix "a" to be transposed, including this listing. The "c" matrix values must also be initialised to zero because of tiling. Multithreading, tiling, and the AVX-512 vector instruction is used. Tiling of block size of eight is required for this implementation. This is a downside of the left shift cyclical rotation over the broadcast shuffle approach. This listing was specifically created for this dissertation, no prior listings were used to create the listing. Some amount of indentations is removed to allow the code to fit into the page. This listing is quite long due to 56 swaps required to swap the *hi* and *lo* accumulator values as explained in Section 3.3.3. Due to how long this listing is only the final version with multithreading is presented with all optimisations that have been determined to perform the best in Chapter 5.

```
void gemm_left_shift_cyclical_rotation_avx_512_tiling_threaded(
        DoubleDouble_SOA_FLAT *a,
        DoubleDouble_SOA_FLAT *b,
        DoubleDouble_SOA_FLAT *c,
        int n) {
    #pragma omp parallel /* Create threads */
    {
        /* The following variables are private
         * to each thread */
        const int block_size = 8;
        __m512d c8_1_hi[block_size];
        __m512d c8_1_lo[block_size];
        #pragma omp for /* Run threads in outer loop */
```

```
for (int ii = 0; ii < n; ii += block_size) {
for (int jj = 0; jj < n; jj += block_size) {
for (int i = 0; i < block_size; i++) {
    c8_1_hi[i] = _mm512_setzero_pd();
    c8_1_lo[i] = _mm512_setzero_pd();
}
for (int kk = 0; kk < n; kk += block_size) {
    for (int k = 0; k < block_size; k++) {
        /* load hi and lo of a matrix */
        __m512d a8_hi =
            _mm512_load_pd(&a->hi[(kk + k) * n + ii]);
        __m512d a8_lo =
            _mm512_load_pd(&a->lo[(kk + k) * n + ii]);

        /* Left shift cyclical rotation of hi vector */
        __m512d a7_hi = _mm512_castsi512_pd(
                        _mm512_alignr_epi64(
                        _mm512_castpd_si512(a8_hi),
                        _mm512_castpd_si512(a8_hi), 7));
        __m512d a6_hi = _mm512_castsi512_pd(
                        _mm512_alignr_epi64(
                        _mm512_castpd_si512(a8_hi),
                        _mm512_castpd_si512(a8_hi), 6));
        __m512d a5_hi = _mm512_castsi512_pd(
                        _mm512_alignr_epi64(
                        _mm512_castpd_si512(a8_hi),
                        _mm512_castpd_si512(a8_hi), 5));
        __m512d a4_hi = _mm512_castsi512_pd(
                        _mm512_alignr_epi64(
                        _mm512_castpd_si512(a8_hi),
                        _mm512_castpd_si512(a8_hi), 4));
        __m512d a3_hi = _mm512_castsi512_pd(
                        _mm512_alignr_epi64(
                        _mm512_castpd_si512(a8_hi),
                        _mm512_castpd_si512(a8_hi), 3));
        __m512d a2_hi = _mm512_castsi512_pd(
                        _mm512_alignr_epi64(
```

```
                    _mm512_castpd_si512(a8_hi),
                    _mm512_castpd_si512(a8_hi), 2));
    __m512d a1_hi = _mm512_castsi512_pd(
                    _mm512_alignr_epi64(
                    _mm512_castpd_si512(a8_hi),
                    _mm512_castpd_si512(a8_hi), 1));

    /* Left shift cyclical rotation of lo vector */
    __m512d a7_lo = _mm512_castsi512_pd(
                    _mm512_alignr_epi64(
                    _mm512_castpd_si512(a8_lo),
                    _mm512_castpd_si512(a8_lo), 7));
    __m512d a6_lo = _mm512_castsi512_pd(
                    _mm512_alignr_epi64(
                    _mm512_castpd_si512(a8_lo),
                    _mm512_castpd_si512(a8_lo), 6));
    __m512d a5_lo = _mm512_castsi512_pd(
                    _mm512_alignr_epi64(
                    _mm512_castpd_si512(a8_lo),
                    _mm512_castpd_si512(a8_lo), 5));
    __m512d a4_lo = _mm512_castsi512_pd(
                    _mm512_alignr_epi64(
                    _mm512_castpd_si512(a8_lo),
                    _mm512_castpd_si512(a8_lo), 4));
    __m512d a3_lo = _mm512_castsi512_pd(
                    _mm512_alignr_epi64(
                    _mm512_castpd_si512(a8_lo),
                    _mm512_castpd_si512(a8_lo), 3));
    __m512d a2_lo = _mm512_castsi512_pd(
                    _mm512_alignr_epi64(
                    _mm512_castpd_si512(a8_lo),
                    _mm512_castpd_si512(a8_lo), 2));
    __m512d a1_lo = _mm512_castsi512_pd(
                    _mm512_alignr_epi64(
                    _mm512_castpd_si512(a8_lo),
                    _mm512_castpd_si512(a8_lo), 1));
```

```
/* load one row of hi and lo of b matrix */
__m512d b8_1_hi =
    _mm512_load_pd(&b->hi[(kk + k) * n + jj]);
__m512d b8_1_lo =
    _mm512_load_pd(&b->lo[(kk + k) * n + jj]);

__m512d temp_hi, temp_lo;

/* Double-Double multiply-add operations for all
 * left shift cyclical rotated vectors */
qd_dd_mul_avx_512(a7_hi, a7_lo, b8_1_hi, b8_1_lo,
                  &temp_hi, &temp_lo);
qd_dd_ieee_add_avx_512_no_store(
        c8_1_hi[0], c8_1_lo[0], temp_hi, temp_lo,
        &c8_1_hi[0], &c8_1_lo[0]);
qd_dd_mul_avx_512(a6_hi, a6_lo, b8_1_hi, b8_1_lo,
        &temp_hi, &temp_lo);
qd_dd_ieee_add_avx_512_no_store(
        c8_1_hi[1], c8_1_lo[1], temp_hi, temp_lo,
        &c8_1_hi[1], &c8_1_lo[1]);
qd_dd_mul_avx_512(a5_hi, a5_lo, b8_1_hi, b8_1_lo,
        &temp_hi, &temp_lo);
qd_dd_ieee_add_avx_512_no_store(
        c8_1_hi[2], c8_1_lo[2], temp_hi, temp_lo,
        &c8_1_hi[2], &c8_1_lo[2]);
qd_dd_mul_avx_512(a4_hi, a4_lo, b8_1_hi, b8_1_lo,
        &temp_hi, &temp_lo);
qd_dd_ieee_add_avx_512_no_store(
        c8_1_hi[3], c8_1_lo[3], temp_hi, temp_lo,
        &c8_1_hi[3], &c8_1_lo[3]);
qd_dd_mul_avx_512(a3_hi, a3_lo, b8_1_hi, b8_1_lo,
        &temp_hi, &temp_lo);
qd_dd_ieee_add_avx_512_no_store(
        c8_1_hi[4], c8_1_lo[4], temp_hi, temp_lo,
        &c8_1_hi[4], &c8_1_lo[4]);
qd_dd_mul_avx_512(a2_hi, a2_lo, b8_1_hi, b8_1_lo,
        &temp_hi, &temp_lo);
```

```
            qd_dd_ieee_add_avx_512_no_store(
                    c8_1_hi[5], c8_1_lo[5], temp_hi, temp_lo,
                    &c8_1_hi[5], &c8_1_lo[5]);
            qd_dd_mul_avx_512(a1_hi, a1_lo, b8_1_hi, b8_1_lo,
                    &temp_hi, &temp_lo);
            qd_dd_ieee_add_avx_512_no_store(
                    c8_1_hi[6], c8_1_lo[6], temp_hi, temp_lo,
                    &c8_1_hi[6], &c8_1_lo[6]);
            qd_dd_mul_avx_512(a8_hi, a8_lo, b8_1_hi, b8_1_lo,
                    &temp_hi, &temp_lo);
            qd_dd_ieee_add_avx_512_no_store(
                    c8_1_hi[7], c8_1_lo[7], temp_hi, temp_lo,
                    &c8_1_hi[7], &c8_1_lo[7]);
        }
    }
    double temp;
    /* Swaps for hi vector values */
    // swap column 0
    temp = c8_1_hi[0][0];
    c8_1_hi[0][0] = c8_1_hi[7][0];
    c8_1_hi[7][0] = temp;

    temp = c8_1_hi[1][0];
    c8_1_hi[1][0] = c8_1_hi[6][0];
    c8_1_hi[6][0] = temp;

    temp = c8_1_hi[2][0];
    c8_1_hi[2][0] = c8_1_hi[5][0];
    c8_1_hi[5][0] = temp;

    temp = c8_1_hi[3][0];
    c8_1_hi[3][0] = c8_1_hi[4][0];
    c8_1_hi[4][0] = temp;

    // swap column 1
    temp = c8_1_hi[1][1];
    c8_1_hi[1][1] = c8_1_hi[7][1];
```

```
c8_1_hi[7][1] = temp;

temp = c8_1_hi[2][1];
c8_1_hi[2][1] = c8_1_hi[6][1];
c8_1_hi[6][1] = temp;

temp = c8_1_hi[3][1];
c8_1_hi[3][1] = c8_1_hi[5][1];
c8_1_hi[5][1] = temp;

// swap column 2
temp = c8_1_hi[0][2];
c8_1_hi[0][2] = c8_1_hi[1][2];
c8_1_hi[1][2] = temp;

temp = c8_1_hi[2][2];
c8_1_hi[2][2] = c8_1_hi[7][2];
c8_1_hi[7][2] = temp;

temp = c8_1_hi[3][2];
c8_1_hi[3][2] = c8_1_hi[6][2];
c8_1_hi[6][2] = temp;

temp = c8_1_hi[4][2];
c8_1_hi[4][2] = c8_1_hi[5][2];
c8_1_hi[5][2] = temp;

// swap column 3
temp = c8_1_hi[0][3];
c8_1_hi[0][3] = c8_1_hi[2][3];
c8_1_hi[2][3] = temp;

temp = c8_1_hi[3][3];
c8_1_hi[3][3] = c8_1_hi[7][3];
c8_1_hi[7][3] = temp;

temp = c8_1_hi[4][3];
```

```
c8_1_hi[4][3] = c8_1_hi[6][3];
c8_1_hi[6][3] = temp;

// swap column 4
temp = c8_1_hi[0][4];
c8_1_hi[0][4] = c8_1_hi[3][4];
c8_1_hi[3][4] = temp;

temp = c8_1_hi[1][4];
c8_1_hi[1][4] = c8_1_hi[2][4];
c8_1_hi[2][4] = temp;

temp = c8_1_hi[4][4];
c8_1_hi[4][4] = c8_1_hi[7][4];
c8_1_hi[7][4] = temp;

temp = c8_1_hi[5][4];
c8_1_hi[5][4] = c8_1_hi[6][4];
c8_1_hi[6][4] = temp;

// swap column 5
temp = c8_1_hi[0][5];
c8_1_hi[0][5] = c8_1_hi[4][5];
c8_1_hi[4][5] = temp;

temp = c8_1_hi[1][5];
c8_1_hi[1][5] = c8_1_hi[3][5];
c8_1_hi[3][5] = temp;

temp = c8_1_hi[5][5];
c8_1_hi[5][5] = c8_1_hi[7][5];
c8_1_hi[7][5] = temp;

// swap column 6
temp = c8_1_hi[0][6];
c8_1_hi[0][6] = c8_1_hi[5][6];
c8_1_hi[5][6] = temp;
```

```
temp = c8_1_hi[1][6];
c8_1_hi[1][6] = c8_1_hi[4][6];
c8_1_hi[4][6] = temp;

temp = c8_1_hi[2][6];
c8_1_hi[2][6] = c8_1_hi[3][6];
c8_1_hi[3][6] = temp;

temp = c8_1_hi[6][6];
c8_1_hi[6][6] = c8_1_hi[7][6];
c8_1_hi[7][6] = temp;

// swap column 7
temp = c8_1_hi[0][7];
c8_1_hi[0][7] = c8_1_hi[6][7];
c8_1_hi[6][7] = temp;

temp = c8_1_hi[1][7];
c8_1_hi[1][7] = c8_1_hi[5][7];
c8_1_hi[5][7] = temp;

temp = c8_1_hi[2][7];
c8_1_hi[2][7] = c8_1_hi[4][7];
c8_1_hi[4][7] = temp;

/* Swaps for lo vector values */
// swap column 0
temp = c8_1_lo[0][0];
c8_1_lo[0][0] = c8_1_lo[7][0];
c8_1_lo[7][0] = temp;

temp = c8_1_lo[1][0];
c8_1_lo[1][0] = c8_1_lo[6][0];
c8_1_lo[6][0] = temp;

temp = c8_1_lo[2][0];
```

```
c8_1_lo[2][0] = c8_1_lo[5][0];
c8_1_lo[5][0] = temp;

temp = c8_1_lo[3][0];
c8_1_lo[3][0] = c8_1_lo[4][0];
c8_1_lo[4][0] = temp;

// swap column 1
temp = c8_1_lo[1][1];
c8_1_lo[1][1] = c8_1_lo[7][1];
c8_1_lo[7][1] = temp;

temp = c8_1_lo[2][1];
c8_1_lo[2][1] = c8_1_lo[6][1];
c8_1_lo[6][1] = temp;

temp = c8_1_lo[3][1];
c8_1_lo[3][1] = c8_1_lo[5][1];
c8_1_lo[5][1] = temp;

// swap column 2
temp = c8_1_lo[0][2];
c8_1_lo[0][2] = c8_1_lo[1][2];
c8_1_lo[1][2] = temp;

temp = c8_1_lo[2][2];
c8_1_lo[2][2] = c8_1_lo[7][2];
c8_1_lo[7][2] = temp;

temp = c8_1_lo[3][2];
c8_1_lo[3][2] = c8_1_lo[6][2];
c8_1_lo[6][2] = temp;

temp = c8_1_lo[4][2];
c8_1_lo[4][2] = c8_1_lo[5][2];
c8_1_lo[5][2] = temp;
```

```
// swap column 3
temp = c8_1_lo[0][3];
c8_1_lo[0][3] = c8_1_lo[2][3];
c8_1_lo[2][3] = temp;

temp = c8_1_lo[3][3];
c8_1_lo[3][3] = c8_1_lo[7][3];
c8_1_lo[7][3] = temp;

temp = c8_1_lo[4][3];
c8_1_lo[4][3] = c8_1_lo[6][3];
c8_1_lo[6][3] = temp;

// swap column 4
temp = c8_1_lo[0][4];
c8_1_lo[0][4] = c8_1_lo[3][4];
c8_1_lo[3][4] = temp;

temp = c8_1_lo[1][4];
c8_1_lo[1][4] = c8_1_lo[2][4];
c8_1_lo[2][4] = temp;

temp = c8_1_lo[4][4];
c8_1_lo[4][4] = c8_1_lo[7][4];
c8_1_lo[7][4] = temp;

temp = c8_1_lo[5][4];
c8_1_lo[5][4] = c8_1_lo[6][4];
c8_1_lo[6][4] = temp;

// swap column 5
temp = c8_1_lo[0][5];
c8_1_lo[0][5] = c8_1_lo[4][5];
c8_1_lo[4][5] = temp;

temp = c8_1_lo[1][5];
c8_1_lo[1][5] = c8_1_lo[3][5];
```

```
c8_1_lo[3][5] = temp;

temp = c8_1_lo[5][5];
c8_1_lo[5][5] = c8_1_lo[7][5];
c8_1_lo[7][5] = temp;

// swap column 6
temp = c8_1_lo[0][6];
c8_1_lo[0][6] = c8_1_lo[5][6];
c8_1_lo[5][6] = temp;

temp = c8_1_lo[1][6];
c8_1_lo[1][6] = c8_1_lo[4][6];
c8_1_lo[4][6] = temp;

temp = c8_1_lo[2][6];
c8_1_lo[2][6] = c8_1_lo[3][6];
c8_1_lo[3][6] = temp;

temp = c8_1_lo[6][6];
c8_1_lo[6][6] = c8_1_lo[7][6];
c8_1_lo[7][6] = temp;

// swap column 7
temp = c8_1_lo[0][7];
c8_1_lo[0][7] = c8_1_lo[6][7];
c8_1_lo[6][7] = temp;

temp = c8_1_lo[1][7];
c8_1_lo[1][7] = c8_1_lo[5][7];
c8_1_lo[5][7] = temp;

temp = c8_1_lo[2][7];
c8_1_lo[2][7] = c8_1_lo[4][7];
c8_1_lo[4][7] = temp;
// Store 8x8 accumulator of hi and 8x8 of lo to c
for (int i = 0; i < block_size; i++) {
```

```
                _mm512_store_pd(&c->hi[(ii + i) * n + jj],
                                c8_1_hi[i]);
                _mm512_store_pd(&c->lo[(ii + i) * n + jj],
                                c8_1_lo[i]);
            }
            }
            }
    }
}
```

Listing 4.12: Left shift cyclical rotation matrix multiplication with multithreading, tiling, and AVX-512. Matrix "a" must be transposed and the "c" matrix values must be initialised to zero before calling this function.

### 4.2.7 Broadcast Shuffle Matrix Multiplication

Listing 4.13 is the first iteration of the broadcast shuffle approach as described in Section 3.3.2. Matrix "a" must be transposed before calling this function. This listing sets the ground work for the optimisations of AVX-512, tiling, two sequential loads of "b" matrix, and multithreading. This function was designed and implemented as part of this dissertation.

```
void gemm_broadcast_shuffle(DoubleDouble_SOA_FLAT *a,
                            DoubleDouble_SOA_FLAT *b,
                            DoubleDouble_SOA_FLAT *c,
                            int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            double c_hi = 0.0;
            double c_lo = 0.0;

            double temp_hi, temp_lo;
            for (int k = 0; k < n; k++) {
                double a_hi = a->hi[k*n + i];
                double a_lo = a->lo[k*n + i];

                double b_hi = b->hi[k*n + j];
                double b_lo = b->lo[k*n + j];
```

```
                qd_dd_mul(a_hi, a_lo,
                          b_hi, b_lo,
                          &temp_hi, &temp_lo);
                qd_dd_ieee_add(c_hi, c_lo,
                               temp_hi, temp_lo,
                               &c_hi, &c_lo);
            }
            c->hi[i*n + j] += c_hi;
            c->lo[i*n + j] += c_lo;
        }
    }
}
```

Listing 4.13: Broadcast shuffle matrix multiplication optimising for register locality as described in Chapter 3. This is the base implementation of the broadcast shuffle algorithm that is optimised in later listings; see Listings 4.15 and 4.16. Matrix "a" must be transposed before calling the function.

### 4.2.8 Broadcast Shuffle AVX-512 Matrix Multiplication

Listing 4.14 modifies Listing 4.13 by updating the code to include AVX-512 compiler intrinsics. The listing has two 8x8 accumulator named *c8_hi* and *c8_lo* for accumulating the double-double multiply-add operations as described in Chapter 3. Matrix "a" must be transposed before calling this function. This function was designed and implemented as part of this dissertation.

```
void gemm_broadcast_shuffle_avx_512(DoubleDouble_SOA_FLAT *a,
                                    DoubleDouble_SOA_FLAT *b,
                                    DoubleDouble_SOA_FLAT *c,
                                    int n) {
    /* 8x8 double-double accumulators */
    __m512d c8_hi[8];
    __m512d c8_lo[8];

    for (int i = 0; i < n; i+=8) {
        for (int j = 0; j < n; j+=8) {
            /* Initialising accumulators to 0 */
```

```
            for (int l = 0; l < 8; l++) {
                c8_hi[l] = _mm512_setzero_pd();
                c8_lo[l] = _mm512_setzero_pd();
            }

            __m512d temp_hi, temp_lo;
            for (int k = 0; k < n; k++) {
                __m512d a8_hi = _mm512_load_pd(&a->hi[k*n + i]);
                __m512d a8_lo = _mm512_load_pd(&a->lo[k*n + i]);

                __m512d b8_hi = _mm512_load_pd(&b->hi[k*n + j]);
                __m512d b8_lo = _mm512_load_pd(&b->lo[k*n + j]);

                for (int l = 0; l < 8; l++) {
                    __m512d a8_hi_l = _mm512_set1_pd(a8_hi[l]);
                    __m512d a8_lo_l = _mm512_set1_pd(a8_lo[l]);

                    /* Double-Double multiply add opeartion */
                    qd_dd_mul_avx_512(a8_hi_l, a8_lo_l,
                                      b8_hi, b8_lo,
                                      &temp_hi, &temp_lo);
                    qd_dd_ieee_add_avx_512_no_store(
                            c8_hi[l], c8_lo[l],
                            temp_hi, temp_lo,
                            &c8_hi[l], &c8_lo[l]);
                }
            }
            /* Storing hi and lo 8x8 accumulator into c */
            for (int l = 0; l < 8; l++) {
                _mm512_store_pd(&c->hi[(i+l)*n + j], c8_hi[l]);
                _mm512_store_pd(&c->lo[(i+l)*n + j], c8_lo[l]);
            }
        }
    }
}
```

Listing 4.14: Broadcast shuffle matrix multiplication optimising for register locality using the AVX-512 vector instruction set. Matrix "a" must be transposed before calling this function.

### 4.2.9 Broadcast Shuffle Additional B Loads Tiling AVX-512 Matrix Multiplication

Listing 4.15 modifies Listing 4.14 by updating the code to use tiling and loading a seuqntial piece of memory from "b" matrix. Matrix "a" must be transposed and the "c" matrix values must be initialised to zero before calling this function. Loading in an addition eight values of "b" matrix requires twice as many double-double multiply-add operations and twice as many register accumulators; see Section 3.3.5. The listing reflects this by introducing two 8x8 accumulators representing one 8x8 double-double accumulator and performing twice as many double-double operations and stores. Three levels of indentation is removed to fit the code snippet into the listing. This function was designed and implemented as part of this dissertation. This code segment is logically equivalent to the previous Listing 4.14 using tiling.

```
void gemm_broadcast_shuffle_avx_512_tiling_additional_b(
        DoubleDouble_SOA_FLAT *a,
        DoubleDouble_SOA_FLAT *b,
        DoubleDouble_SOA_FLAT *c,
        int n) {
    /* 8x8 Double-Double accumulator */
    __m512d c8_1_hi[8];
    __m512d c8_1_lo[8];

    /* 8x8 Double-Double accumulator */
    __m512d c8_2_hi[8];
    __m512d c8_2_lo[8];
    const int block_size = 64;

    /* Tiling optimisation:
     * Three for loops itearting in blocks */
    for (int ii = 0; ii < n; ii += block_size) {
    for (int jj = 0; jj < n; jj += block_size) {
    for (int kk = 0; kk < n; kk += block_size) {
```

```
int i_cond = (ii + block_size < n) ?
             (ii + block_size) : n;
int j_cond = (jj + block_size < n) ?
             (jj + block_size) : n;
/* For loops use ii, jj, kk indicies
 * to iterate in sections (blocks) of matrices */
for (int i = ii; i < i_cond; i+=8) {
    for (int j = jj; j < j_cond; j+=16) {/* 2 b loads */
        for (int l = 0; l < 8; l++) {
            c8_1_hi[l] =
                _mm512_load_pd(&c->hi[(i+l)*n + j]);
            c8_1_lo[l] =
                _mm512_load_pd(&c->lo[(i+l)*n + j]);
            c8_2_hi[l] =
                _mm512_load_pd(&c->hi[(i+l)*n + (j+8)]);
            c8_2_lo[l] =
                _mm512_load_pd(&c->lo[(i+l)*n + (j+8)]);
            __m512d temp_hi, temp_lo;
            for (int k = kk; k < kk + block_size &&
                             k < n; k++) {
                __m512d a8_hi =
                    _mm512_load_pd(&a->hi[k*n + i]);
                __m512d a8_lo =
                    _mm512_load_pd(&a->lo[k*n + i]);
                /* load eight double-double
                 * elements of b */
                __m512d b8_1_hi =
                    _mm512_load_pd(&b->hi[k*n + j]);
                __m512d b8_1_lo =
                    _mm512_load_pd(&b->lo[k*n + j]);
                /* load next eight double-double
                 * elements of b */
                __m512d b8_2_hi =
                    _mm512_load_pd(&b->hi[k*n + (j+8)]);
                __m512d b8_2_lo =
                    _mm512_load_pd(&b->lo[k*n + (j+8)]);
                for (int l = 0; l < 8; l++) {
```

```
            __m512d a8_hi_l =
                _mm512_set1_pd(a8_hi[l]);
            __m512d a8_lo_l =
                _mm512_set1_pd(a8_lo[l]);
            /* Double-Double multiply-add
             * operation on
             * b8_1_hi and b8_1_lo */
            qd_dd_mul_avx_512(
                    a8_hi_l, a8_lo_l,
                    b8_1_hi, b8_1_lo,
                    &temp_hi, &temp_lo);
            qd_dd_ieee_add_avx_512_no_store(
                    c8_1_hi[l], c8_1_lo[l],
                    temp_hi, temp_lo,
                    &c8_1_hi[l], &c8_1_lo[l]);

            /* Double-Double multiply-add
             * operation on next eight
             * elements of b  */
            qd_dd_mul_avx_512(
                    a8_hi_l, a8_lo_l,
                    b8_2_hi, b8_2_lo,
                    &temp_hi, &temp_lo);
            qd_dd_ieee_add_avx_512_no_store(
                    c8_2_hi[l], c8_2_lo[l],
                    temp_hi, temp_lo,
                    &c8_2_hi[l], &c8_2_lo[l]);
        }
        for (int l = 0; l < 8; l++) {
            /* Store the double-double 8x8
             * accumulator into c */
            _mm512_store_pd(
                    &c->hi[(i+l)*n + j],
                    c8_1_hi[l]);
            _mm512_store_pd(
                    &c->lo[(i+l)*n + j],
                    c8_1_lo[l]);
```

```
                        /* Store the next double-double
                         * 8x8 accumulator into
                         * next values of c */
                        _mm512_store_pd(
                                &c->hi[(i+l)*n + (j+8)],
                                 c8_2_hi[l]);
                        _mm512_store_pd(
                                &c->lo[(i+l)*n + (j+8)],
                                 c8_2_lo[l]);
                    }
                }
            }
        }
    }
    }
    }
}
```

Listing 4.15: Broadcast shuffle matrix multiplication optimising for register locality using the AVX-512 vector instruction set, tiling, and loading an additional eight values from the "b" matrix. Matrix "a" must be transposed and the "c" matrix values must be initialised to zero before calling this function.

### 4.2.10 Broadcast Shuffle Additional B Load Multithreaded Tiling AVX-512 Matrix Multiplication

Listing 4.16 modifies Listing 4.15 by updating the code to use multithreading. This is the final iteration of the broadcast shuffle approach as described in Chapter 3, Section 3.3.2. This register locality optimised algorithm implements optimisations of AVX-512, tiling, multithreading, and loading an additional row from the "b" matrix. Listings 4.15 and 4.16 use four 8x8 accumulators making up two 8x8 double-double accumulators. The total number of double-double multiply-add operations is doubled for use with the additional row of "b", and the AVX-512 store instructions store the additional 8x8 double-double accumulator. Tiling introduces the need to reload values of matrix "c" compared to Listings 4.13, and 4.14 which zero the accumulators each loop. Matrix "a" must

be transposed and the "c" matrix values must be initialised to zero before calling this function. This function was designed and implemented as part of this dissertation.

```
void gemm_broadcast_shuffle_avx_512_tiling_additional_b_threaded(
        DoubleDouble_SOA_FLAT *a,
        DoubleDouble_SOA_FLAT *b,
        DoubleDouble_SOA_FLAT *c,
        int n) {
    #pragma omp parallel
    {
        __m512d c8_1_hi[8];
        __m512d c8_1_lo[8];

        __m512d c8_2_hi[8];
        __m512d c8_2_lo[8];
        const int block_size = 64;
        #pragma omp for
        for (int ii = 0; ii < n; ii += block_size) {
        for (int jj = 0; jj < n; jj += block_size) {
        for (int kk = 0; kk < n; kk += block_size) {
            int i_cond = (ii + block_size < n) ?
                         (ii + block_size) : n;
            int j_cond = (jj + block_size < n) ?
                         (jj + block_size) : n;
            for (int i = ii; i < i_cond; i+=8) {
                for (int j = jj; j < jj + block_size &&
                                    j < n; j+=16) {
                    for (int l = 0; l < 8; l++) {
                        c8_1_hi[l] =
                            _mm512_load_pd(&c->hi[(i+l)*n + j]);
                        c8_1_lo[l] =
                            _mm512_load_pd(&c->lo[(i+l)*n + j]);
                        c8_2_hi[l] =
                            _mm512_load_pd(&c->hi[(i+l)*n
                                        + (j+8)]);
                        c8_2_lo[l] =
                            _mm512_load_pd(&c->lo[(i+l)*n
                                        + (j+8)]);
```

```
}

__m512d temp_hi, temp_lo;
for (int k = kk; k < kk + block_size &&
                    k < n; k++) {
    __m512d a8_hi =
        _mm512_load_pd(&a->hi[k*n + i]);
    __m512d a8_lo =
        _mm512_load_pd(&a->lo[k*n + i]);

    __m512d b8_1_hi =
        _mm512_load_pd(&b->hi[k*n + j]);
    __m512d b8_1_lo =
        _mm512_load_pd(&b->lo[k*n + j]);

    __m512d b8_2_hi =
        _mm512_load_pd(&b->hi[k*n + (j+8)]);
    __m512d b8_2_lo =
        _mm512_load_pd(&b->lo[k*n + (j+8)]);

    for (int l = 0; l < 8; l++) {
        __m512d a8_hi_l =
            _mm512_set1_pd(a8_hi[l]);
        __m512d a8_lo_l =
            _mm512_set1_pd(a8_lo[l]);

        qd_dd_mul_avx_512(
                a8_hi_l, a8_lo_l,
                b8_1_hi, b8_1_lo,
                &temp_hi, &temp_lo);
        qd_dd_ieee_add_avx_512_no_store(
                c8_1_hi[l], c8_1_lo[l],
                temp_hi, temp_lo,
                &c8_1_hi[l], &c8_1_lo[l]);

        qd_dd_mul_avx_512(
                a8_hi_l, a8_lo_l,
```

```
                                        b8_2_hi, b8_2_lo,
                                        &temp_hi, &temp_lo);
                                qd_dd_ieee_add_avx_512_no_store(
                                        c8_2_hi[l], c8_2_lo[l],
                                        temp_hi, temp_lo,
                                        &c8_2_hi[l], &c8_2_lo[l]);
                            }
                        }
                        for (int l = 0; l < 8; l++) {
                            _mm512_store_pd(&c->hi[(i+l)*n + j],
                                            c8_1_hi[l]);
                            _mm512_store_pd(&c->lo[(i+l)*n + j],
                                            c8_1_lo[l]);

                            _mm512_store_pd(&c->hi[(i+l)*n + (j+8)],
                                            c8_2_hi[l]);
                            _mm512_store_pd(&c->lo[(i+l)*n + (j+8)],
                                            c8_2_lo[l]);
                        }
                    }
                }

            }
            }
            }
        }
}
```

Listing 4.16: Final iteration of the broadcast shuffle matrix multiplication algorithm, optimised for register locality using AVX-512, tiling, multithreading, and loading an additional values of "b" matrix. Matrix "a" must be transposed and the "c" matrix values must be initialised to zero before calling this function.

## 4.3 Summary

This chapter introduced the modifications required for double-double operations using the AVX-512 vector instruction set. The chapter gave general snippets of code for setting up the data layout for matrix multiplication. Finally the chapter gave code segments with varying optimisations, some code segments contained AVX-512, tiling, multithreading, loading additional rows of "b" matrix and some did not. Code for a register locality optimised algorithm was designed and implemented specfically for this dissertation in Listings 4.12, 4.13, 4.14, 4.15, and 4.16. The general matrix multiplication that this dissertation calls "j_major" was not designed for this dissertation, but was modified to use double-double operations and a combination of tiling, and multithreading. The "j_major" implementations are found in Listings 4.8, 4.9, 4.10, and 4.11. An objective of this dissertation was to explore different implementations including existing implementations. The existing implementations were refined and were modified to use double-double operations operations.

# Chapter 5

# Evaluation

This chapter presents the runtime performance of the matrix multiplication algorithms from Chapter 4 in seconds, as seen in Listings 5.1 and 5.2, the clock cycles per double-double multiply-add operation in Listings 5.5 and 5.6, and the nano seconds per double-double multiply-add operation in Listings 5.3 and 5.4. All measurements were collected on a Tiger Lake 11th generation Intel Core i7-1165G7, 2.8GHz base processor, a 4 core, 8 threaded processor. Figure 5.7 illustrates that 4 threads performs better than 8 threads for this processor.

## 5.1 Measuring Time

In the implementation Chapter 4, Listings 4.7, 4.8, 4.9, 4.10, 4.11, 4.12, 4.13, 4.14, 4.15, and 4.16 are presented. The time execution of the matrix multiplication algorithms were measured using the "gettimeofday" function in C from the "sys/time.h" standard header library. The difference between the two time stamps was the total time taken for the algorithm to execute.

```
#include <stdlib.h>
#include <sys/time.h>
struct timeval start_time, stop_time;
gettimeofday(&start_time, NULL);
/* algorithm to evaluate runtime performance */
gettimeofday(&stop_time, NULL);
long long time_microseconds =
    (stop_time.tv_sec - start_time.tv_sec) * 1000000L +
    (stop_time.tv_usec - start_time.tv_usec);
```

Listing 5.1: C code for measuring runtime in microseconds.

Intel x86/x64 processors maintain a hardware counter that tracks the number of processor cycles since the processor was last reset. This counter can be accessed with the Read Time-Stamp Counter (RDTSC) instruction [22]. Previously in this dissertation clock cycles were discussed in relation to AVX-512 intrinsics in Section 2.1.1. C requires inline assembly to call the RDTSC instruction. GCC provides Extended Asm [23] for calling assembly instructions in C. The Listing 5.2 provides the code necessary for creating an inlined function that returns the clock cycles.

```
#include <stdint.h>
static inline uint64_t rdtsc(void) {
    uint32_t hi, lo;
    __asm__ __volatile__ ("rdtsc" : "=a"(lo), "=d"(hi));
    return (((uint64_t)hi)<<32) | ((uint64_t)lo);
}
```

Listing 5.2: C inline assembly code for calling the RDTSC instruction. The EDX:EAX registers form a combined 64 bits of the resulting clock cycles [22]. The values of EDX and EAX are stored in *hi* and *lo* 32 bit integers respectively and bit manipulated to form a 64 bit result.

## 5.2 Measuring Threaded Time

OpenMP includes a function to change the number of threads created for execution in multithreaded functions. The function "void omp_set_num_threads(int num_threads)" takes in an integer indicating the number of threads to create. In Figure 5.7, the Listings 4.12, 4.11, and 4.16 used a different amount of threads marked by the number of threads created in the suffix of the names in the legend.

## 5.3 Experiments

Experiments were performed on square double-double precision matrices of sizes 128x128, 256x256, 512x512, 1024x1024, and 2048x2048. The clock cycles and execution time are graphed in Figures 5.1, 5.3, 5.5, and 5.7. All functions in the graphs were named after their respective code implementations from Section 4.2.

## 5.4 Results

Two matrix multiplication algorithms were determined to be efficient as compared to the algorithms as shown in Figures 5.1, 5.3, and 5.5. The register locality optimised matrix multiplication called "gemm_broadcast_shuffle_avx_512_tiling_additional_b_threaded"; see Listing 4.16 and Figure 5.7 was determined to perform best with 4 threads in runtime performance measured in seconds. The matrix multiplication algorithms "gemm_j_major" were not designed for this dissertation, instead modifications were made taking the original source work [21] and using double-double precision operations, tiling, and multithreading.

### 5.4.1 Execution Time in Seconds

Figure 5.1 graphs the execution time in seconds for square matrices of different sizes for the implementations in Section 4.2. This graph illustrates that the AVX-512 vector instruction set, tiling, and threading improves performance. The slowest algorithms, "gemm" and "gemm_broadcast_shuffle" were the algorithms which did not use AVX-512, multithreading, tiling, and multithreading. The rest of the implementations in between 0 and 13 seconds are multithreaded and can be viewed in a zoomed in figure; reference Figure 5.1 for an explanation of the faster implementations.

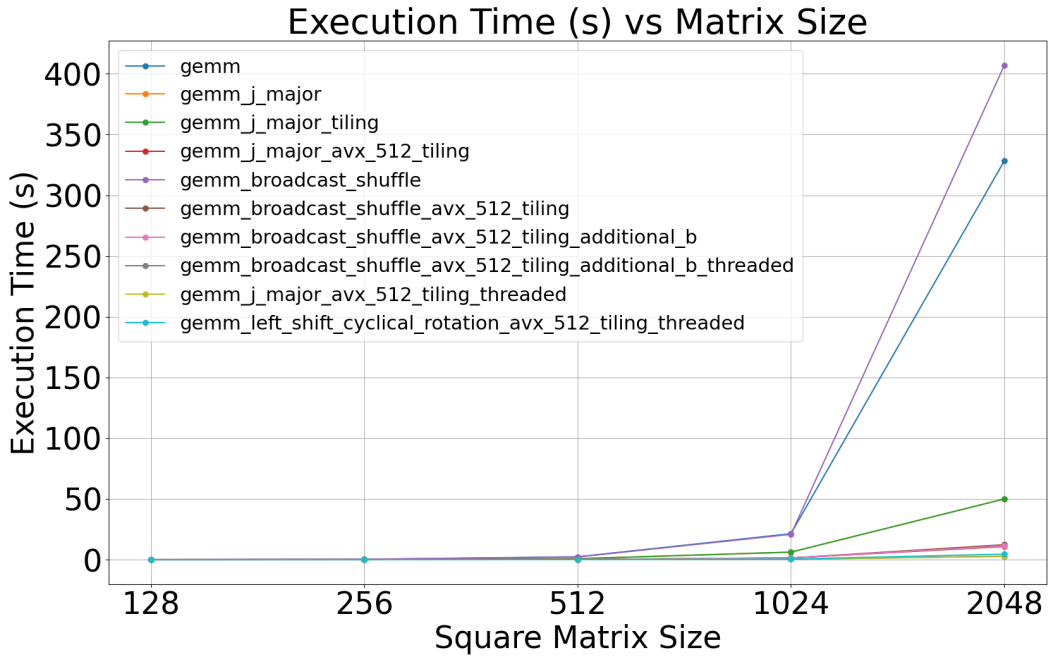


Figure 5.1: The execution time in seconds of different sized double-double square matrices. The three fastest algorithms were multithreaded.

Figure 5.2 is the zoomed in illustration of Figure 5.1. The implementations "gemm_left_shift_cyclical_rotation" and "gemm_broadcast_shuffle" implementations were designed and implemented specifically for this dissertation. The "j_major" algorithms were modified with double-double precision, tiling, and multithreading. The design of "j_major" is described in source [21].
"gemm_broadcast_shuffle_avx_512_tiling_additional_b_threaded" takes approximately 2.8 seconds, "gemm_left_shift_cyclical_rotation_avx_512_tiling_threaded" takes approximately 4.5 seconds and "gemm_major_avx_512_tiling_threaded" takes approximately 2.6 seconds for a 2048x2048 matrix.

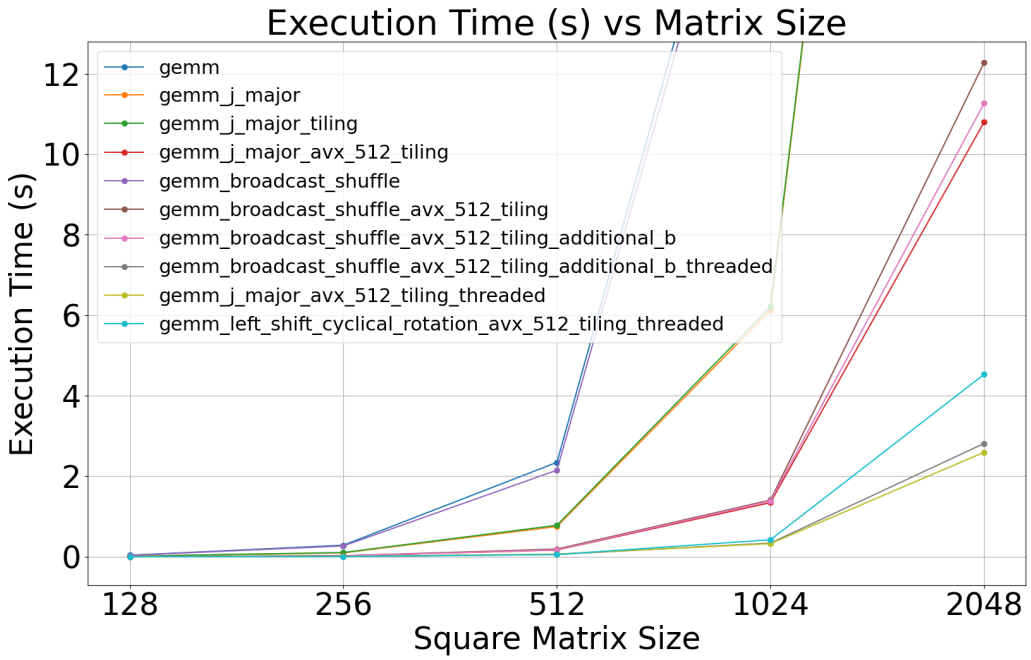


Figure 5.2: Zoomed in representation of Figure 5.1. The faster algorithms ranging from 0 to 13 seconds on 2048x2048 use optimisations from register locality, tiling, AVX-512, addition b values, and multithreading. Listing 4.11 runs in the least amount of time and does not use register locality or loading of additional b values. A close second in performance is the algorithm designed and implemented for this dissertation optimising register locality is Listing 4.16.

### 5.4.2 Execution Time per Multiply-Add

Figure 5.3 graphs the execution time in nanoseconds per double-double multiply-add operation. A double-double multiply-add operation is calculated dividing the time taken by $N^3$. The matrix multiplication implementations for this dissertation are all of time complexity $O(N^3)$, where $N^3$ amounts to the total instructions performed. The graph represents the constant factor of one instruction for each implementation. All the implementations apart from "gemm" and "gemm_broadcast_shuffle" scale expectedly with different sized matrices. The unoptimised implementations require more time per instruction as the size of the matrix gets larger. Reference Figure 5.4 for a zoomed in representation of the graph between 0 and 150 nanoseconds.

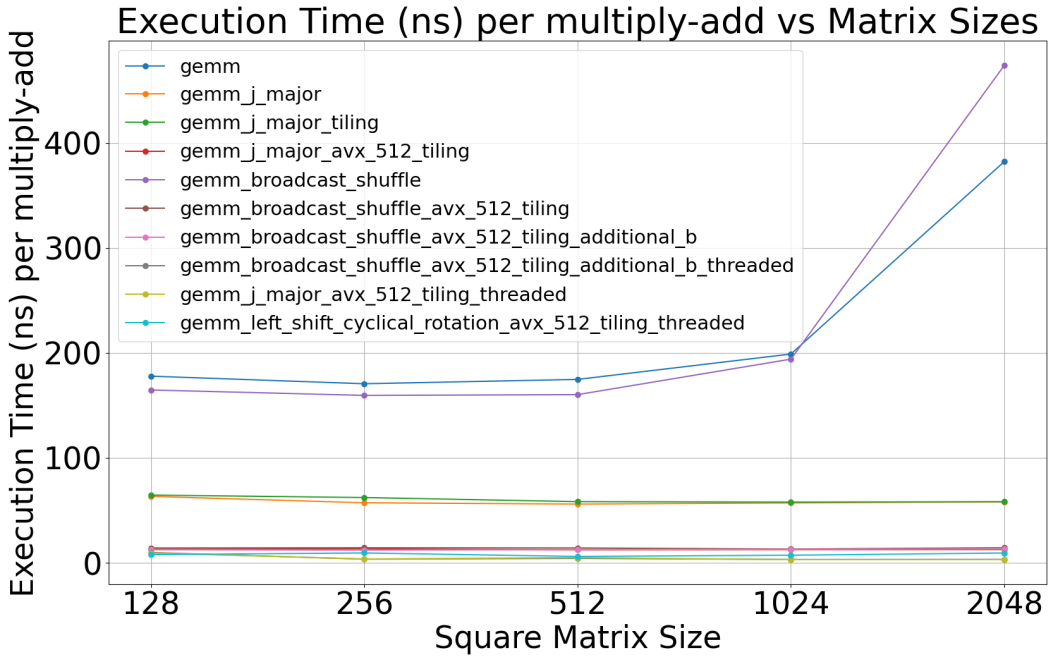


Figure 5.3: The execution time in nanoseconds per double-double multiply-add operation of different sized square matrices. The algorithms that are efficient take the same amount of time per multiply-add given a larger matrix unlike the inefficient algorithms.

Figure 5.4 provides a close-up view of Figure 5.3. The close-up is between 0 and 30 nanoseconds. Similarly to Figure 5.1 the three matrix multiplication algorithms that perform better than the rest are "gemm_broadcast_shuffle_avx_512_tiling_additional_b_threaded", "gemm_left_shift_cyclical_rotation_avx_512_tiling_threaded",
and "gemm_j_major_avx_512_tiling_threaded". The results of the three implementations is an execution time ranging approximately 3 nanoseconds to a little over 5 nanoseconds per double-double multiply-add operation.

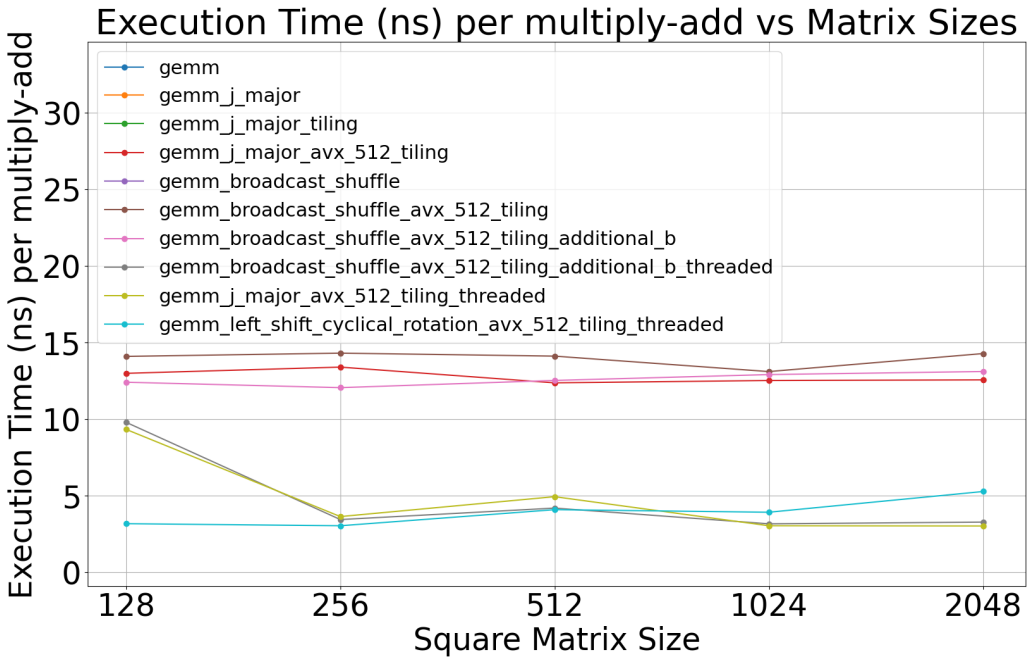


Figure 5.4: Zoomed in representation of Figure 5.3. The faster algorithms ranging from 0 to 30 nanoseconds on 2048x2048 use optimisations from register locality, tiling, AVX-512, loading additional b value and multithreading.

### 5.4.3 Clock Cycles per Multiply-Add

Figure 5.5 graphs the total clock cycles per double-double multiply-add operation. Similarly to Figure 5.3 the number of cycles divided by $N^3$ operations calculates the clock cycles per multiply-add. In Figure 5.6, a close-up view of the faster implementations between 0 and 35 clock cycles per double-double multiply-add. Clock cycles are not graphed for multithreaded implementations because threads execute concurrently on different CPU cores, each with its own counter. The RDTSC instruction would not reflect the performance of an algorithm because it is not designed to display the total clock cycles on all threads.

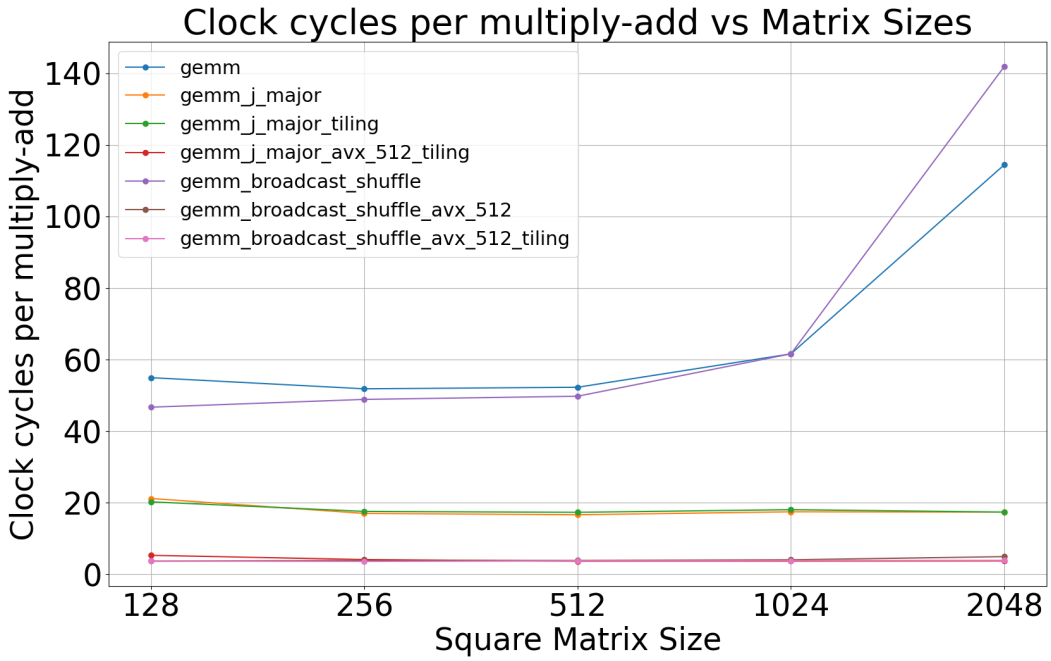


Figure 5.5: Clock cycles per double-double multiply-add operation of different sized square matrices. All AVX-512 optimised matrix multiplication implementations are between 0 and 25 clock cycles per multiply-add operation.

Figure 5.6 is a close-up of Figure 5.5. The clock cycles are divided by $N^3$ to determine clock cycles per double-double multiply-add operation. The fastest implementations are between 3 and 6 clock cycles per multiple-add operation. The theoretical limit assuming no pipelining or latency and that each floating point instruction takes 1 clock cycle is 28 clock cycles per eight double-double multiply-add. "gemm_broadcast_shuffle_avx_512_tiling" performs approximately 3.93 cycles per double-double operation. $3.93 \times 8$ results in 31.44 clock cycles for eight double-double operations. "gemm_j_major_avx_512_tiling" performs approximately 3.763 cycles per double-double operation. Therefore $3.763 \times 8$ results in 30.1 clock cycles for eight double-double operations. The theoretical limit of 28 cycles is almost reached which is what makes the implementations so efficient.

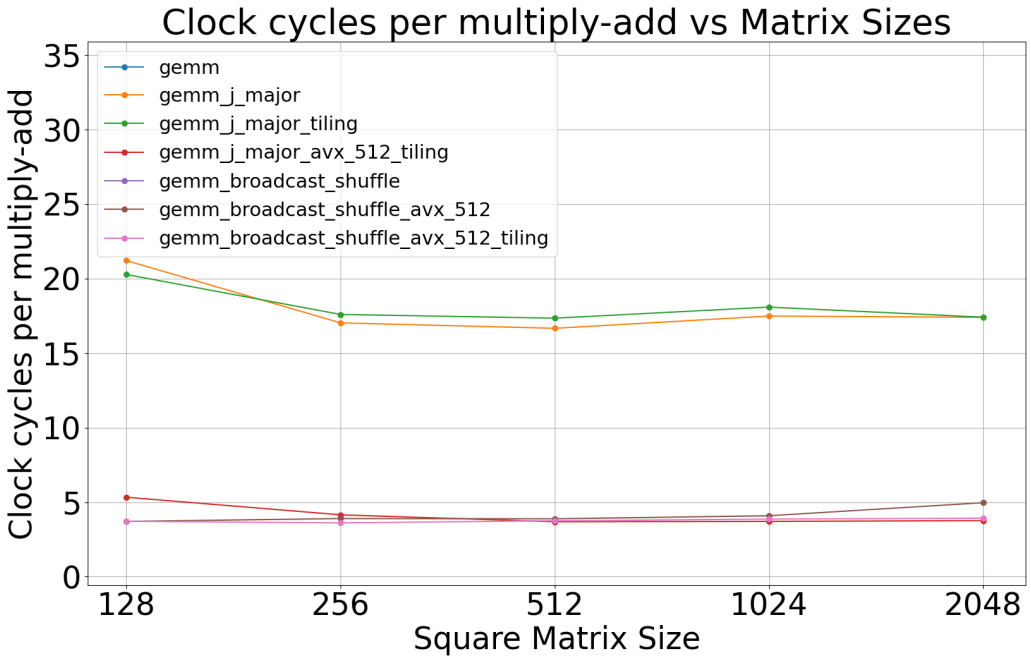


Figure 5.6: Zoomed in representation of Figure 5.5. The theoretical limit of 28 clock cycles is almost reached with the two different implementations taking 30.1 and 31.44 cycles per AVX-512 double-double operation, and 3.763 and 3.93 cycles per single double-double operation.

### 5.4.4 Different Number of Threads

Figure 5.7 compares the different number of threads and how if effects the execution speed measured in seconds for the three different matrix multiplication designs from this dissertation. "gemm_left_shift_cyclical_rotation_avx_512_tiling_threaded" executes fastest with 8 threads. The other two implementations execute faster with 4 threads. Refer to Figure 5.8 for the execution time of the three separate designed algorithms.

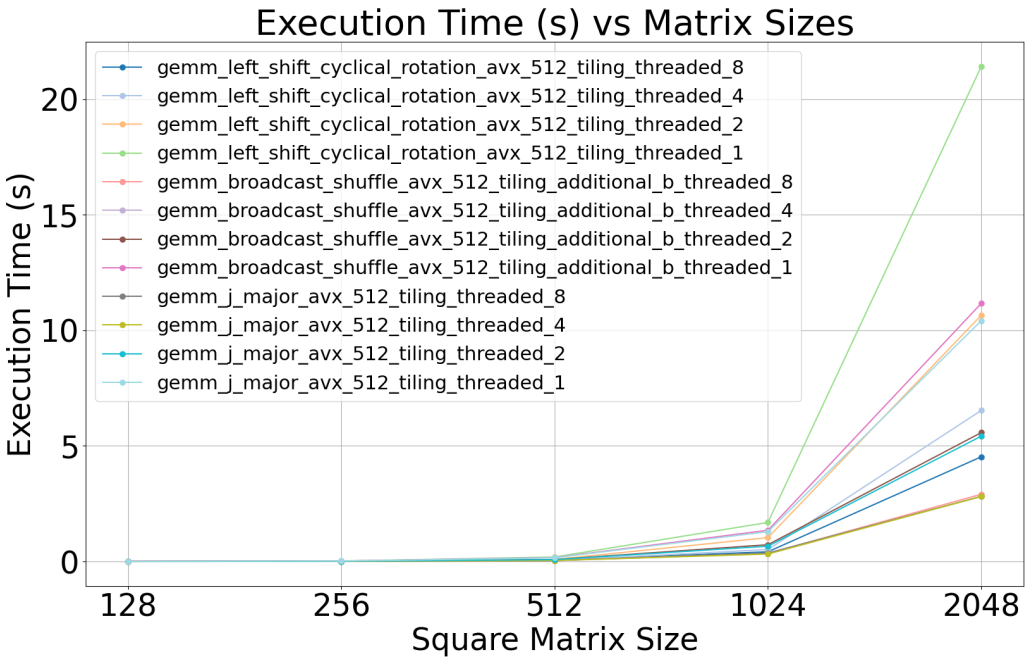


Figure 5.7: Matrix multiplication algorithms optimised for register locality 4.16 and 4.14 are compared with 1, 2, 4, and 8 threads. Tiger Lake 11th generation Intel Core i7-1165G7, 2.8 GHz processor, a 4 core, 8 threaded processor is most effective with 4 threads.

Figure 5.8 represents the zoomed in view of Figure 5.7 between 0 and 10 seconds. ‘gemm_j_major_avx_512_tiling_threaded_4” and “gemm_broadcast_shuffle_avx_512_tiling_additional_b_threaded_4” executes in approximately 2.81 seconds. The matrix multiplication implementation, “gemm_left_shift_cyclical_rotation_avx_512_tiling_threaded_8” executes in approximately 4.5 seconds.

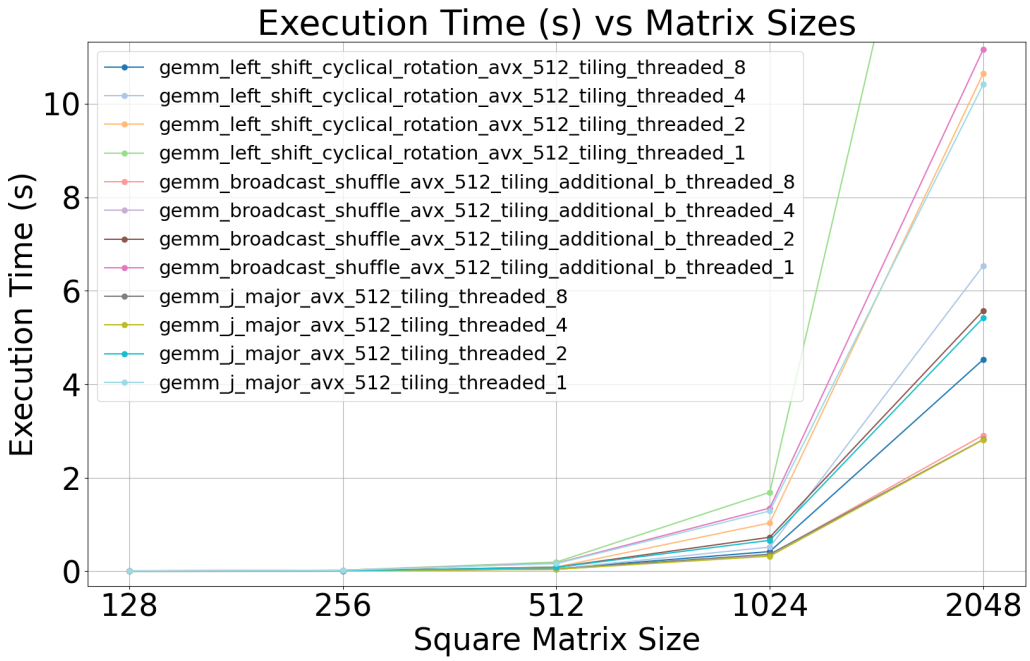


Figure 5.8: Zoomed in representation of Figure 5.7. The two fastest implementations of broadcast shuffle and j_major best with 4 threads.

### 5.4.5 Matrix Multiplication Timings

Table 5.1 displays the execution time of matrix multiplication implementations as discussed in Chapters 4 and 5. The execution time is provided in microseconds for different sized square matrices. The following results are provided in Figures 5.1, 5.2, 5.3, 5.4, 5.7, and 5.8.

| **Matrix multiplication algorithms** | **128x128** | **256x256** | **512x512** | **1024x1024** | **2048x2048** |
|---|---|---|---|---|---|
| gemm | 37273 | 286090 | 2344195 | 21345602 | 328169454 |
| gemm_j_major | 13261 | 95926 | 750445 | 6124196 | 49914210 |
| gemm_j_major_tiling | 13521 | 104286 | 781588 | 6209167 | 49943099 |
| gemm_left_shift_cyclical_rotation_avx_512_tiling_threaded | 3854 | 22413 | 188139 | 1325180 | 10797513 |
| gemm_broadcast_shuffle | 34510 | 267463 | 2149524 | 20824833 | 406864735 |
| gemm_broadcast_shuffle_avx_512_tiling_additional_b | 2603 | 20230 | 168170 | 1386466 | 11263352 |
| gemm_broadcast_shuffle_avx_512_tiling_additional_b_threaded | 2054 | 5773 | 56246 | 339586 | 2813281 |
| gemm_j_major_tiling_threaded | 1957 | 6100 | 66243 | 326020 | 2594471 |

Table 5.1: Execution times in microseconds for different matrix multiplication algorithms and matrix sizes.

Table 5.2 displays the execution time in clock cycles of different matrix multiplication implementations and sizes from Chapters 4 and 5. The clock cycles were recorded using the RDTSC instruction, which provides an accurate timing of single-threaded execution. Multithreaded implementations are excluded from this table because the execution overlaps with no way of calculating the total cycles over all threads. An execution on more than one thread of execution has no way of determining the cycles per instruction. Sequential execution provides a reliable way to determine the total clock cycles through processing instructions one after the other but multithreaded execution does not. The following results were used to determine the clock cycles per double-double operation in Figures 5.5 and 5.6.

| **Matrix multiplication algorithms** | **128x128** | **256x256** | **512x512** | **1024x1024** | **2048x2048** |
|---|---|---|---|---|---|
| gemm | 115269304 | 869943074 | 7017477850 | 66148381530 | 982932963956 |
| gemm_j_major | 44478744 | 285764722 | 2237083418 | 18780693636 | 149503009840 |
| gemm_j_major_tiling | 42522316 | 295246144 | 2328333248 | 19423488816 | 149589536660 |
| gemm_j_major_avx_512_tiling | 11183876 | 69670828 | 494598476 | 3985370744 | 32330491186 |
| gemm_broadcast_shuffle | 97970880 | 820266562 | 6681123934 | 66164703554 | 1218641024920 |
| gemm_broadcast_shuffle_avx_512 | 7780618 | 65456110 | 521545614 | 4391957546 | 42644521764 |
| gemm_broadcast_shuffle_avx_512_tiling | 7797274 | 60592672 | 503699034 | 4152737346 | 33735986000 |

Table 5.2: Execution times in clock cycles for different matrix multiplication algorithms and matrix sizes.

# Chapter 6

# Conclusions & Future Work

This dissertation has studied double-double precision [5] and referenced existing double-double algorithms of "Quick-Two-Sum", "Two-Sum", and "Two-Prod". The algorithms were used as helper algorithms for defining double-double addition and multiplication functions named "qd_dd_ieee_add" and "qd_dd_mul"; see Listings 2.1, 2.2, 2.3, 2.4, and 2.5. The following listings were modified to use the AVX-512 vector instruction set; see Listings 4.1, 4.2, 4.3, 4.4, and 4.6. Two matrix multiplication algorithms were designed for this dissertation. The two designs are the broadcast shuffle and the left shift cyclical rotation matrix multiplication algorithms. The algorithms are described with detail in Sections 3.3.2 and 3.3.3. The j_major matrix multiplication implementations in this dissertation are taken from source [21]. The j_major matrix multiplication were modified to use the double-double operations and optimisations of tiling and multithreading; see Listings 4.8, 4.9, 4.10, and 4.11. The broadcast shuffle and left shift cyclical rotation implementations optimised for register locality. The j_major implementation did not. The final iterations of the broadcast shuffle implementation is "gemm_broadcast_shuffle_avx_512_tiling_threaded", it executes in 3.275 nanoseconds per multiply-add operation. "gemm_broadcast_shuffle_avx_512_tiling" executes in 3.93 clock cycles per double-double operation, making the calculation for eight double-double operations execute in 31.44 cycles for a 512 bit wide vector in AVX-512.
"gemm_j_major_avx_512_tiling_threaded" executes in 3.02 nanoseconds per double-double multiply add operation. "gemm_j_major_avx_512_tiling" executes in 3.763 clock cycles per double-double multiply-add operation, making the 512 bit wide vector execute in 30.1 clock cycles. The execution time of "gemm_left_shift_cyclical_rotation" is 5.27 nanoseconds per double-double multiply-add operation. The clock cycles are not provided for the left shift cyclical rotation matrix multiplication because only a threaded implementation is presented in this dissertation. The matrix multiplication design of broadcast shuffle was faster than the left shift cyclical rotation. In conclusion the most efficient implementation

is “gemm_j_major_avx_512_tiling_threaded” with
“gemm_broadcast_shuffle_avx_512_tiling_threaded” being a close second, it executed 117 times faster than “gemm”.

In Chapter 1, the research objectives for this dissertation were provided alongside the research question. The research questions asked what optimisations make the most effective matrix multiplication algorithm. The following optimisations were discussed and implemented:

- Data layout of structure of arrays
- Aligning memory
- Contiguous memory
- Register locality
- Loading in additional elements of the second matrix.
- AVX-512
- Tiling
- Multithreading

This dissertation has completed the five objectives that were set out from the research objectives in Section 1.3:

1. All matrix multiplication implementations use double-double precision; see Listings 4.7, 4.8, 4.9, 4.10, 4.11, 4.12, 4.13, 4.15, and 4.16.
2. Listings 4.1, 4.2, 4.3, 4.4, and 4.6 implement AVX-512 equivalent double-double operations.
3. The impact was discussed in Chapter 3, and the conclusion was to use a structure of arrays layout for double-double precision matrices.
4. Listings 4.8, 4.9, 4.10, and 4.16 are existing j_major implementations of matrix multiplications modified to use double-double precision, and additional optimisations of threading, and tiling.
5. In Chapter 5, AVX-512 was an instruction set that improved performance. Listings 4.12, 4.13, 4.15, and 4.16 use the additional registers from the instruction set to implement a register locality implementation of matrix multiplication.

## 6.1 Future Work

This section discusses the work that is beyond the scope of this dissertation. I also reference the limitations of my solutions and further improvements that could be made.

### 6.1.1 Matrix Multiplication Limitations

Register locality optimised matrix multiplication algorithms which use AVX-512 as shown in Listings 4.12, 4.15, and 4.16 require certain dimensions be divisible by eight. The dimension which does not require divisibility by eight is the $k$ dimension or in other words the number rows of both matrices. The algorithm as discussed in Section 3.3 by design allows for any number of rows. For dimensions $i$ and $j$ the restriction of divisibility by eight is needed to execute reasonably efficient operations without needing to add additional conditions in the algorithm and a new set of instructions. The 512 bit wide vectors from the AVX-512 vector instruction set load eight values at a time which is where the divisibility by eight comes from. A valid solution to the restriction is to pad a matrix that is not divisible by eight along the $i$ and $j$ dimensions with zeros. Matrix multiplication that is padded out with zeros has no effect on the other values of the matrix after matrix multiplication. Some future work would be to implement such an algorithm as a utility for the set of matrix multiplication functions described in Chapter 4.

### 6.1.2 Explore the impact of AVX-512 on different processors

Future work exploring the different performance impacts of AVX-512 on different architectures. This dissertation analysed the performance of the Tiger Lake 11th generation Intel Core i7-1165G7, 2.8GHz processor. Compiler optimised code from GCC and Clang prefers 256 bit wide vectors over 512 bit wide vectors due to the frequency drops found in certain Intel architectures like Sky Lake and Tiger Lake [24]. Compiling C code with auto vectorisation enabled suggests that AMD zen4/zen5 architectures prefer the 512 bit wide vectors. A Comparison between AMD's zen4/zen5 architecture and the Tiger Lake architecture used for this dissertation may provide valuable insights on the limitations of AVX-512 on different architectures.

### 6.1.3 Further Evaluation

Comparing different sized matrices and tiling size in matrix multiplication is additional work that can be done. Converting existing highly efficient matrix multiplication algorithms to double-double precision matrix multiplication algorithms and comparing them against the implementations presented in this dissertation may provide insights to other optimisations that are used in matrix multiplication algorithms.

### 6.1.4 Updating code to use AVX10

In the introduction, I mentioned that Intel announced their upcoming vector instruction set called AVX10 [9]. This instruction set has optional 512 bit wide vectors. For hardware implementations that do not support 512 bits, the algorithms listings described in this dissertation utilising AVX-512 must be adjusted to use 256 bit wide vectors instead. Speed improvements are promised with AVX10 and the compatibility with AVX-512 is high as all existing functions of AVX-512 256 bits and below are forward compatible with AVX10. For AVX10 which has 512 bit support, no further adjustments to the algorithms discussed in this dissertation are required.

### 6.1.5 GPU implementations of double-double precision matrix multiplication

Research on double-double matrix multiplication for an NVIDIA C2050 GPU [25] exists. GPU programming is different from CPU programming and does not support SIMD like AVX-512. However, the speed improvements that GPUs provide are high. Double-Double precision lends itself well for GPU algorithms due to the lack of branching for computing a double-double precision number. Future work researching platform independent solutions for double-double precision GPU matrix multiplication would be relevant to the many GPUs like AMD and Intel which do not support CUDA (Compute Unified Device Architecture). Research on creating an OpenCL (Open Computing Language) solution [26], an open cross-platform parallel computing programming for GPUs is one example of work that is relevant to this research field.

### 6.1.6 Arbitrary precision matrix multiplication

Floating point numbers can be represented as any arbitrary precision number. AVX-512 can be used to scale to arbitrary precision computation as well. In this dissertation I represented double-double computation using two summations of doubles, a *hi*, and a *lo* value. The structure of arrays data layout can be modified to hold additional higher precision matrices; see Figure 3.2. With arbitrary precision this concept can be extended to represent more double values by utilising more vectors for the next bits which are unable to fit in the *lo* double. The register locality optimised algorithm in Listing 4.16 would have to be adjusted by loading in eight rather than sixteen values of "b" due to the restriction of 32 registers per core. Existing libraries such as MPFR [3] have arbitrary precision computation which do not perform matrix multiplication, but may provide a useful resource for researching arbitrary precision resources. The QD library [5] provides code for performing quad-double precision, this is a summation of four double precision numbers, representing 256 bits. Functions from the QD library for performing quad-double precision floating point computations can be extended similarly as described in this dissertation.

# Bibliography


[1] Intel, "Intel intrinsics guide," *Intel Document*, 2024. [Online]. Available: https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html#

[2] H. Saeed and M. A. Nazari, "Applications of matrix multiplication," *Journal for Research in Applied Sciences and Biotechnology*, 2024. [Online]. Available: https://api.semanticscholar.org/CorpusID:270275368

[3] T. M. Team, "The gnu mpfr library," https://www.mpfr.org/, 2025, version 4.2.1.

[4] H. Hasegawa. (2003, 01) Utilizing the quadruple-precision floating-point arithmetic operation for the krylov subspace methods.

[5] X. S. L. Yozo Hida and D. H. Bailey, *Library for Double-Double and Quad-Double Arithmetic*. University of California, Berkeley, May 2008, no. LBNL-47918. [Online]. Available: https://www.davidhbailey.com/dhbpapers/qd.pdf

[6] ——, *Github Library for Double-Double and Quad-Double Arithmetic*. University of California, Berkeley, May 2008, no. LBNL-47918. [Online]. Available: https://github.com/scibuilder/QD

[7] Intel, "What is intel avx-512," *Intel Document*, 2016. [Online]. Available: https://www.intel.com/content/www/us/en/products/docs/accelerator-engines/what-is-intel-avx-512.html

[8] X. Amberger, "Intel completely disables avx-512 on alder lake after all – questionable interpretation of "efficiency"," *Igorslab*, 2021. [Online]. Available: https://www.igorslab.de/en/intel-deactivated-avx-512-on-alder-lake-but-fully-questionable-interpretation-of-efficiency-news-editorial/

[9] Intel, "The converged vector isa: Intel advanced vector extension 10," *Intel Document*, p. 1, 2023.

[10] AMD, "4th gen amd epyc processor architecture," *AMD Document*, vol. 4, p. 9, 2023. [Online]. Available: https://www.amd.com/content/dam/amd/en/documents/epyc-business-docs/white-papers/221704010-B_en_4th-Gen-AMD-EPYC-Processor-Architecture---White-Paper_pdf.pdf

[11] ——, "5th gen amd epyc processor architecture," *AMD Document*, vol. 1, p. 8, 2024. [Online]. Available: https://www.amd.com/content/dam/amd/en/documents/epyc-business-docs/white-papers/5th-gen-amd-epyc-processor-architecture-white-paper.pdf

[12] GCC, "Gcc options for c," *GCC GNU Website*, 2025. [Online]. Available: https://gcc.gnu.org/onlinedocs/gcc/C-Dialect-Options.html

[13] LLVM, "Floating point support in clang," *LLVM release documentation*, 2025. [Online]. Available: https://releases.llvm.org/14.0.0/tools/clang/docs/ReleaseNotes.html#floating-point-support-in-clang

[14] GCC, "x86 options," *GCC GNU Website*, 2025. [Online]. Available: https://gcc.gnu.org/onlinedocs/gcc/x86-Options.html

[15] IEEE Standards Association, "Ieee standard for floating-point arithmetic," *IEEE Std 754-2019 (Revision of IEEE 754-2008)*, pp. 1–84, 2019.

[16] J. Dumas, C. Pernet, and A. Sedoglavic, "Strassen's algorithm is not optimally accurate," in *Proceedings of the 2024 International Symposium on Symbolic and Algebraic Computation (ISSAC)*. ACM, 2024, pp. 254–263. [Online]. Available: https://doi.org/10.1145/3666000.3669697

[17] M. S. Lam, E. E. Rothberg, and M. E. Wolf, "The cache performance and optimization of blocked algorithms," in *Proceedings of the Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-IV)*. Santa Clara, CA, USA: ACM, 1991, pp. 63–74.

[18] Intel, "Cache blocking techniques," *Intel Document*, 2019. [Online]. Available: https://www.intel.com/content/www/us/en/developer/articles/technical/cache-blocking-techniques.html

[19] The Linux man-pages project, *pthreads(7) - Linux manual page*, 2024, accessed: 2025-04-10. [Online]. Available: https://man7.org/linux/man-pages/man7/pthreads.7.html

[20] OpenMP Architecture Review Board, *OpenMP Application Programming Interface Examples Version 6.0*, OpenMP Architecture Review Board, Nov. 2024. [Online]. Available: https://www.openmp.org/wp-content/uploads/openmp-examples-6.0.pdf

[21] C. Jarvis, "Simd and avx512," https://www.uio.no/studier/emner/matnat/ifi/IN3200/v19/teaching-material/avx512.pdf, 2019, lecture notes for IN3200, University of Oslo.

[22] F. Cloutier, "Rdtsc — read time-stamp counter," 2025. [Online]. Available: https://www.felixcloutier.com/x86/rdtsc

[23] GNU Project, *Extended Asm - Assembler Instructions with C Expression Operands*, GCC Online Documentation. [Online]. Available: https://gcc.gnu.org/onlinedocs/gcc/Extended-Asm.html

[24] LLVM Project, "Limit number of vector registers used for skylake and icelake," https://reviews.llvm.org/D67259, 2019, discussion on limiting vector register usage to 256 for Skylake-AVX512, Icelake-Client, and Icelake-Server architectures, following GCC's lead. Excludes KNL.

[25] M. Nakata, Y. Takao, S. Noda, and R. Himeno, "A fast implementation of matrix-matrix product in double-double precision on nvidia c2050 and application to semidefinite programming," in *2012 Third International Conference on Networking and Computing*, 2012, pp. 68–75.

[26] The Khronos Group, "OpenCL - The open standard for parallel programming of heterogeneous systems," https://www.khronos.org/opencl/, 2024.